

EEE 475 - Medical Imaging Reconstruction and Processing

Homework 3

Tuna Şahin, 22201730

Contents

0	Introduction	2
0.1	GenAI Policy	2
1	Question 1	2
1.1	Question 1.1	2
1.2	Question 1.2	3
1.3	Question 1.3	4
2	Question 2	5
2.1	Question 2.1	5
2.2	Question 2.2	6
2.3	Question 2.3	7
2.4	Question 2.4	8
2.5	Question 2.5	11
2.6	Question 2.6	12
2.7	Question 2.7	13
2.8	Question 2.8	15
3	Question 3	17
3.1	Question 3.1	17
3.2	Question 3.2	18
3.3	Question 3.3	21
3.4	Question 3.4	23
3.5	Question 3.5	27
3.6	Question 3.6	29
3.7	Question 3.7	31
4	Question 4	34
4.1	Question 4.1	34
4.2	Question 4.2	35
4.3	Question 4.3	36
4.4	Question 4.4	37
5	Conclusion	42

Introduction

In this homework, we will be working with different MPI simulation results, measurements, and reconstruction techniques such as Singular Value Decomposition (SVD) and Kachmarz method. The goal is to analyze the effects of different parameters on the reconstruction quality such as condition number, regularization parameters, number of iterations, and noise levels. We will obtain the PSNR and SSIM values along with visual comparisons of the reconstructions and error maps to evaluate the performance of the reconstruction methods under various conditions. Guided by these metrics, we will look at different approaches for determining the regularization parameter. We will initially work with a simulated dataset and then apply the same techniques to a real dataset OpenMPI [1].

GenAI Policy

ChatGPT 5.4 was used to brainstorm about the validity of the results. Coding was mostly done manually with the exception of lines [465,468] where again ChatGPT 5.4 was used. VSCode Copilot assisted in writing the latex code along with automatic image captions for the report. All remaining technical implementation, analysis, and interpretation of the results were done without the use of GenAI tools.

Question 1

This question asks us to process and view the system matrix for use in the upcoming questions. We will be reshaping the system matrix, along with the measurement to fit our needs and obtain the reference image.

Question 1.1

We will load the system matrix from the provided .mat file [2]. This provides us with a $2 \times 1632 \times 2500$ tensor. We will then visualise the recordings for three arbitrary locations and coils. These arbitrary locations were chosen as (1, 5), (2, 131), and (1, 1051) where the first index denotes the coil number and the second index denotes the location. The system matrix rows for these locations and coils are shown in Figure 1.1.1. Since the MPI measurements are real-valued concentrations, the system matrix rows are conjugate symmetric. This means that half of the values in the system matrix rows are redundant and can be inferred from the other half.

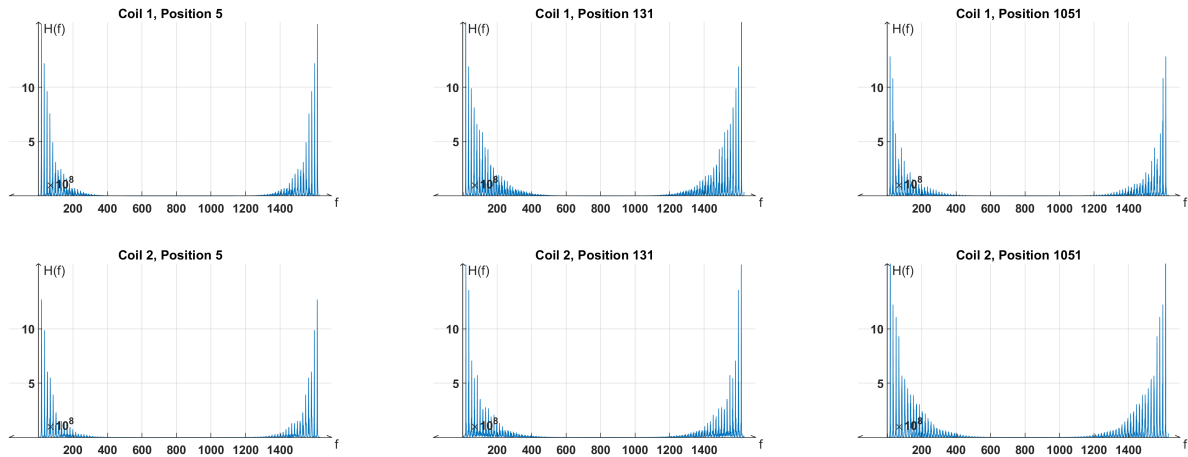


Figure 1.1.1: System Matrix Rows for Three Arbitrary Locations and Coils.

```

1 data = load("SM_25nm_50x50.mat");
2 SM = data.SM;
3 figure;
4 subplot(2,3,1)
5 fineplot(1:1632,abs(SM(1,:,5)),"Coil 1, Position 5","f","H(f)",[-170 1700],[0
  ↳ 16*(10^8)],"off",[2000 400],"","-");
6 subplot(2,3,2)
7 fineplot(1:1632,abs(SM(1,:,131)),"Coil 1, Position 131","f","H(f)",[-170 1700],[0
  ↳ 16*(10^8)],"off",[2000 400],"","-");
8 subplot(2,3,3)
9 fineplot(1:1632,abs(SM(1,:,1051)),"Coil 1, Position 1051","f","H(f)",[-170 1700],[0
  ↳ 16*(10^8)],"off",[2000 400],"","-");
10 subplot(2,3,4)
11 fineplot(1:1632,abs(SM(2,:,5)),"Coil 2, Position 5","f","H(f)",[-170 1700],[0
  ↳ 16*(10^8)],"off",[2000 400],"","-");
12 subplot(2,3,5)
13 plot(abs(SM(2,:,131)))
14 fineplot(1:1632,abs(SM(2,:,131)),"Coil 2, Position 131","f","H(f)",[-170 1700],[0
  ↳ 16*(10^8)],"off",[2000 400],"","-");
15 subplot(2,3,6)
16 fineplot(1:1632,abs(SM(2,:,1051)),"Coil 2, Position 1051","f","H(f)",[-170 1700],[0
  ↳ 16*(10^8)],"off",[2000 400],"","-");
17 set(gcf,'Units','Normalized','OuterPosition',[0,0,1,0.6]);
18 if save_image
19     exportgraphics(gcf, strcat("q11_sysmatrows.png"), 'Resolution', 300)
20 end

```

Question 1.2

As discussed, the system matrix is conjugate symmetric, which means that we can omit half of the values without losing any information. Also to flatten the system matrix down to 2 dimensions, we will be concatenating the matrices for each coil into a single matrix. Finally, we will separate the real and imaginary parts and concatenate them to obtain a real-valued system matrix. The resulting system matrix will have dimensions of (3264×2500) . This process decreased the number of elements in the system from $2 \times 1632 \times 2500 = 8,160,000$ to $3264 \times 2500 = 8,160,000$, which is the same number of elements but now the elements are real-valued instead of complex-valued. Since complex numbers have twice the dimensionality of real numbers, we can say that we have effectively halved the number of data in the system matrix. The file size for these two matrices will have also decreased from 116,058 kB to 61,591 kB, which is effectively halving the file size considering the additions of file headers and metadata.

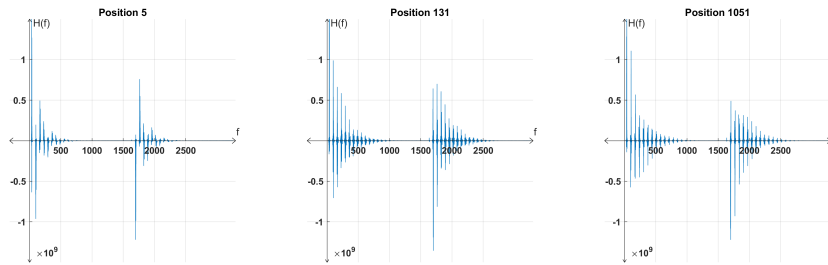


Figure 1.2.1: System Matrix Rows for Three Arbitrary Locations and Coils after Reshaping.

```

1 S = SM(:,1:end/2,:);
2 S = reshape(S,2*816,2500);
3 S = [real(S); imag(S)];
4 figure;
5 subplot(1,3,1)
6 fineplot(1:3264,S(:,5),"Position 5","f","H(f)",[-320 3300],[-1.5
  ↳ 1.5]*(10^9),"off",[1500 400],"","-");
7 subplot(1,3,2)
8 fineplot(1:3264,S(:,131),"Position 131","f","H(f)",[-320 3300],[-1.5
  ↳ 1.5]*(10^9),"off",[1500 400],"","-");
9 subplot(1,3,3)
10 fineplot(1:3264,S(:,1051),"Position 1051","f","H(f)",[-320 3300],[-1.5
  ↳ 1.5]*(10^9),"off",[1500 400],"","-");
11 set(gcf, 'Units', 'Normalized', 'OuterPosition', [0, 0, 1, 0.5]);
12 if save_image
13     exportgraphics(gcf,strcat("q12_sysmatrows.png"),'Resolution',300)
14 end
15 save("S.mat","S")

```

Question 1.3

We will load the measurement from the provided .mat file [3]. This provides us with a 2×1632 matrix. We will then apply similar processing to this matrix to obtain a real-valued measurement vector of size (3264×1) . A plot of the measurement vector is shown in Figure 1.3.1.

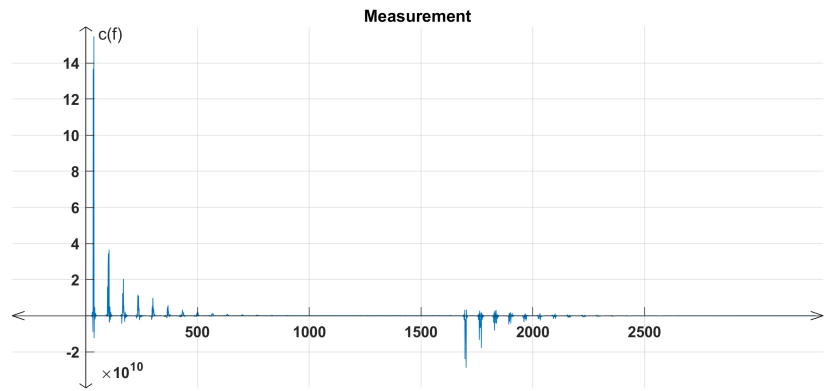


Figure 1.3.1: Measurement Vector after Reshaping

Within the same file we also have the reference image, which is a 50×50 matrix. Also, the maximum value of the reference image is calculated and all of the following images in Questions 1,2, and 3 will be plotted with the interval $[0max_val]$ and the error maps will be plotted with the interval $[0max_val/2]$ to allow for easier comparisons between different techniques and parameters. The reference image is shown in Figure 1.3.2.

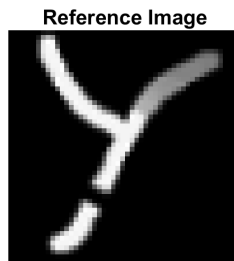


Figure 1.3.2: Reference Image

```

1 data = load("meas_25nm_phantom.mat");
2 m_ref = data.phantom_ref;
3 meas = data.meas;
4 figure;
5 imshow(m_ref,[])
6 title("Reference Image")
7 max_val = max(m_ref(:));
8 set(gcf, 'Units', 'Normalized', 'OuterPosition', [0, 0, 0.5, 0.5]);
9 if save_image
10     exportgraphics(gcf, strcat("q13_ref.png"), 'Resolution', 300)
11 end
12 u = meas(:,1:end/2);
13 u = reshape(u, 2*816, 1);
14 u = [real(u); imag(u)];
15 figure;
16 fineplot(1:length(u), u, "Measurement", "f", "c(f)", [-330 3300], [-4
    ↪ 16]*(10^10), "off", [800 600], "", "-");
17 set(gcf, 'Units', 'Normalized', 'OuterPosition', [0, 0, 0.6, 0.45]);
18 if save_image
19     exportgraphics(gcf, strcat("q13_meas.png"), 'Resolution', 300)
20 end

```

Question 2

This question focuses on the reconstruction of the image using Singular Value Decomposition (SVD) and different methods of processing the singular values to obtain the reconstruction. These processes include thresholding and regularization.

Question 2.1

We use MATLAB's built-in SVD function to decompose the system matrix into its singular values and vectors. The largest singular value is found to be 1.6791×10^{11} , while the smallest singular value is found to be 1.8009×10^{-6} . The condition number of the system matrix is calculated as $\frac{\sigma_{max}}{\sigma_{min}} = \frac{1.6791 \times 10^{11}}{1.8009 \times 10^{-6}} = 9.3222 \times 10^{16}$. MATLAB's built-in condition number function gives us 5.9066×10^{17} . This discrepancy is probably due to numerical precision issues when calculating the smallest singular value, which is very close to zero. To test this hypothesis, the digit precision of MATLAB was increased from 32 to 64 with no change in the results. Then the online MATLAB environment was tested with the identical file and lines. The results obtained were different from the results obtained previously. This suggests that CPU architecture differences or other hardware related factors may be influencing the results and hence small deviances may occur from person to person. The plot of the singular values is shown in Figure 2.1.1. The flat reigon in the plot of the singular values indicates possible numerical precision issues as well.

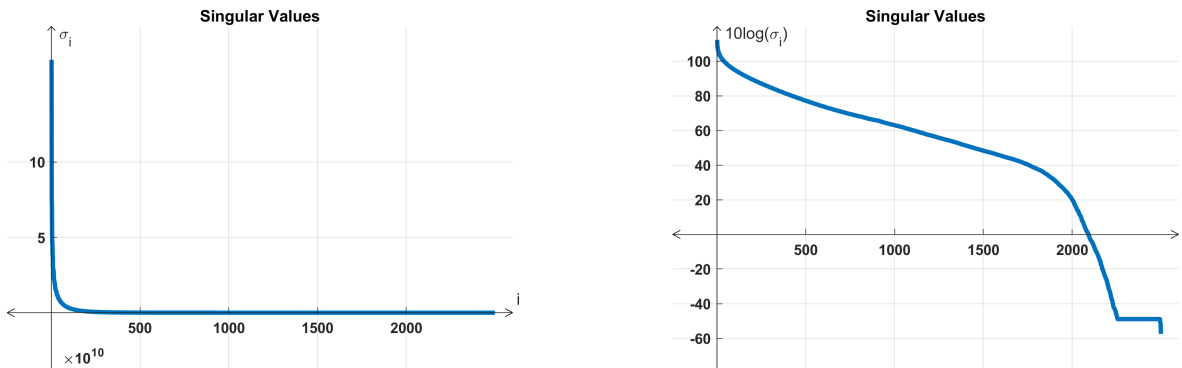


Figure 2.1.1: Singular Values of the System Matrix

```

1 S = load("S.mat");
2 S = S.S;
3 [U,Sigma,V] = svd(S,'econ');
4 cond_num = Sigma(1,1)/Sigma(end,end)
5 cond_num2 = cond(S);
6 fprintf("Condition Number from Singular Values : %s, Condition Number from MATLAB
   ↳ Function: %s",num2sci(cond_num),num2sci(cond_num2))
7 fprintf("Largest Singular Value: %s, Smallest Singular Value:
   ↳ %s",num2sci(Sigma(1,1)),num2sci(Sigma(end,end)))
8 figure;
9 subplot(1,2,1)
10 p = fineplot(1:2500,sum(Sigma),"Singular Values","n","\sigma",[-250 2600],[-4
   ↳ 19]*(10^10),"off",[800 400],"","-");
11 p.LineWidth = 3;
12 subplot(1,2,2)
13 p = fineplot(1:2500,10*log10(sum(Sigma)),"Singular Values","n","10log(\sigma)",[-250
   ↳ 2600],[-80 120],"off",[1200 400],"","-");
14 p.LineWidth = 3;
15 set(gcf, 'Units', 'Normalized', 'OuterPosition', [0, 0, 0.45, 0.45]);
16 if save_image
17     exportgraphics(gcf, strcat("q21_sigma.png"), 'Resolution', 300)
18 end

```

Question 2.2

We can then use the Moore-Penrose pseudoinverse to obtain the reconstruction of the image. We can directly obtain the reconstruction by multiplying the pseudoinverse of the system matrix with the measurement vector. The resulting reconstruction is shown in Figure 2.2.1.

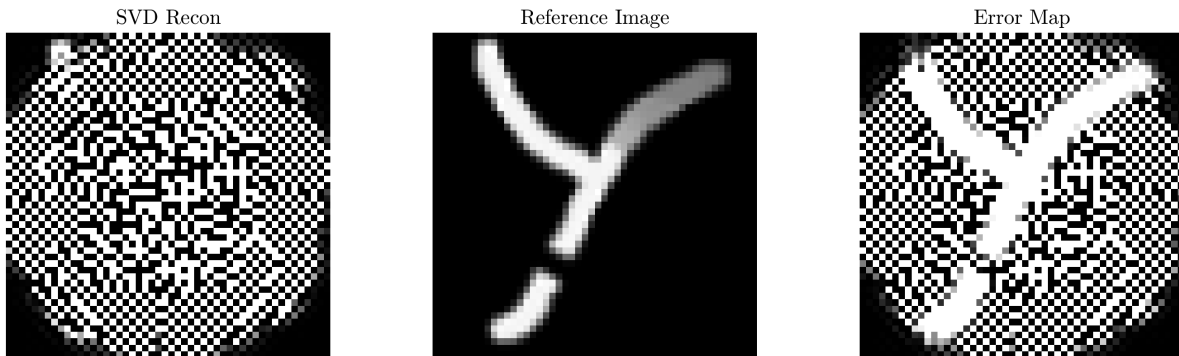


Figure 2.2.1: Reconstruction using Moore-Penrose Pseudoinverse

$$\text{SSIM} = 0.0330, \text{PSNR} = -120.5297$$

The resulting image does not resemble the reference image at all. This observation is supported by extremely low SSIM and PSNR values which are 0.0330 and -120.5297 respectively. This is because the system matrix is extremely ill-conditioned, which means that small changes in the measurement vector or small numerical precision errors can lead to large changes in the reconstruction.

```

1  s = diag(Sigma);
2  sinv = zeros(size(s));
3  tol = 1e-12; % or use a tolerance based on max(s)
4  sinv(s > tol) = 1 ./ s(s > tol);
5  Sigma_inv = diag(sinv);
6  c = V * Sigma_inv * ctranspose(U) * u;
7  ima = reshape(c,50,50);
8  ima(ima<0) = 0;
9  plot_comparative_images(ima,m_ref,save_image,"q22_comp.png","SVD Recon","Reference
  ↳ Image",max_val)
10 PSNR2_2 = psnr(ima/max_val,m_ref/max_val);
11 SSIM2_2 = ssim(ima/max_val,m_ref/max_val);

```

Question 2.3

We can delete small singular values and the rows and columns corresponding to them to obtain a more stable reconstruction. We can choose a threshold value for the singular values and omit those that are below the threshold. The desired condition number is chosen as 100. There are $M = 143$ rows that meet this criteria. This results in a more reasonable reconstruction, as shown in Figure 2.3.1. However, the reconstruction still does not resemble the reference image very well. This is because we are still omitting a large number of singular values, which means that we are losing a lot of information about the system.

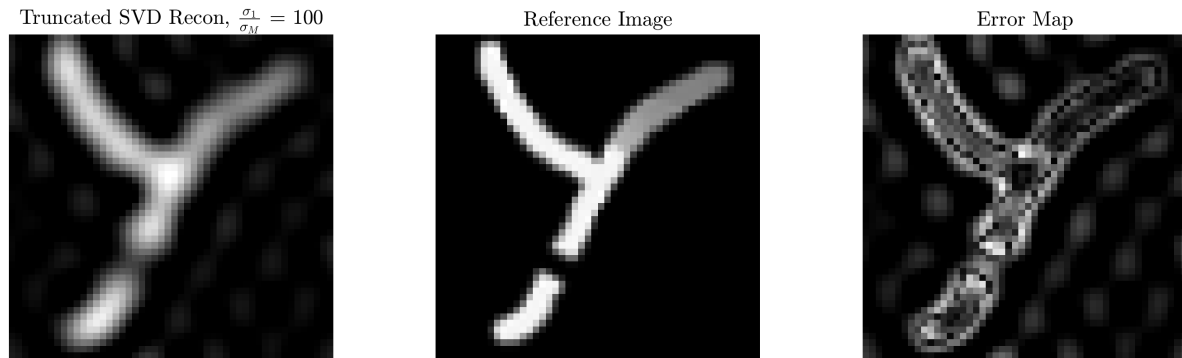


Figure 2.3.1: Reconstruction with Thresholding

SSIM = 0.4219 , PSNR = 19.7741

```

1  cond_num_desired = 100;
2  [val,M] = min(abs(s-(s(1)/cond_num_desired)));
3  Sigma_new = Sigma(1:M,1:M);
4  U_new = U(:,1:M);
5  V_new = V(:,1:M);
6  Sigma_new_inv = invert_sigma(Sigma_new);
7  c = V_new * Sigma_new_inv * ctranspose(U_new) * u;
8  ima = reshape(c,50,50);
9  ima(ima<0) = 0;
10 plot_comparative_images(ima,m_ref,save_image,"q23_comp.png",strcat("Truncated SVD
  ↳ Recon,  $\frac{\sigma_1}{\sigma_M} =$ ",num2str(cond_num_desired)),"Reference
  ↳ Image",max_val)
11 PSNR2_3 = psnr(ima/max_val,m_ref/max_val);
12 SSIM2_3 = ssim(ima/max_val,m_ref/max_val);

```

Question 2.4

We can change the threshold value to see how it affects the reconstruction. We will experiment with 9 different threshold values, which correspond to condition numbers of 1, 5, 10^1 , 10^2 , 10^3 , 10^4 , 10^5 , 10^6 , and 10^7 . These reconstructions are shown in Figures 2.4.1 through 2.4.9.

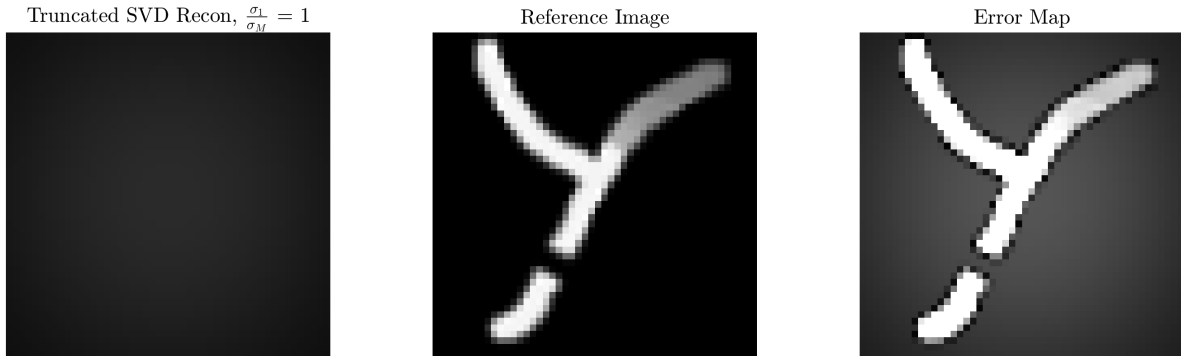


Figure 2.4.1: Reconstruction with Thresholding for Condition Number of 1

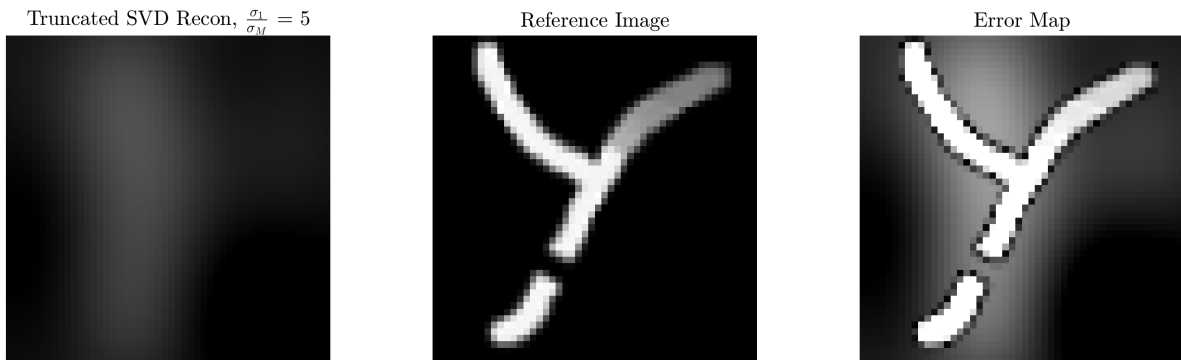


Figure 2.4.2: Reconstruction with Thresholding for Condition Number of 5

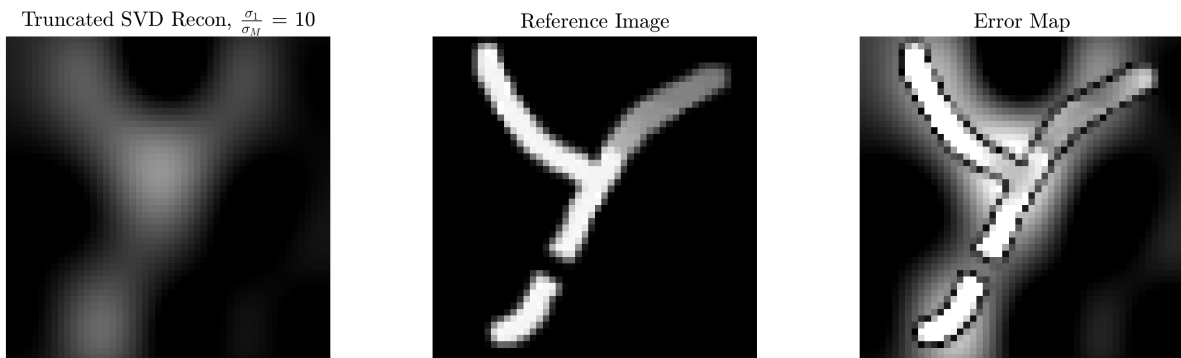


Figure 2.4.3: Reconstruction with Thresholding for Condition Number of 10

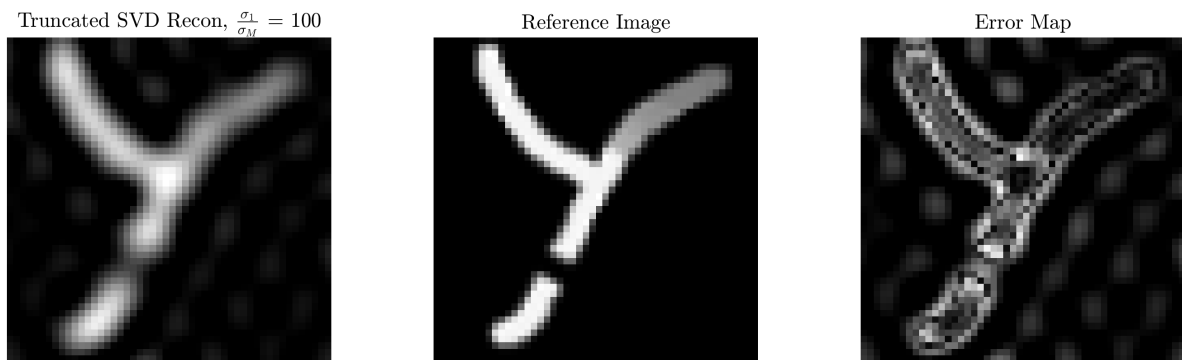


Figure 2.4.4: Reconstruction with Thresholding for Condition Number of 10^2

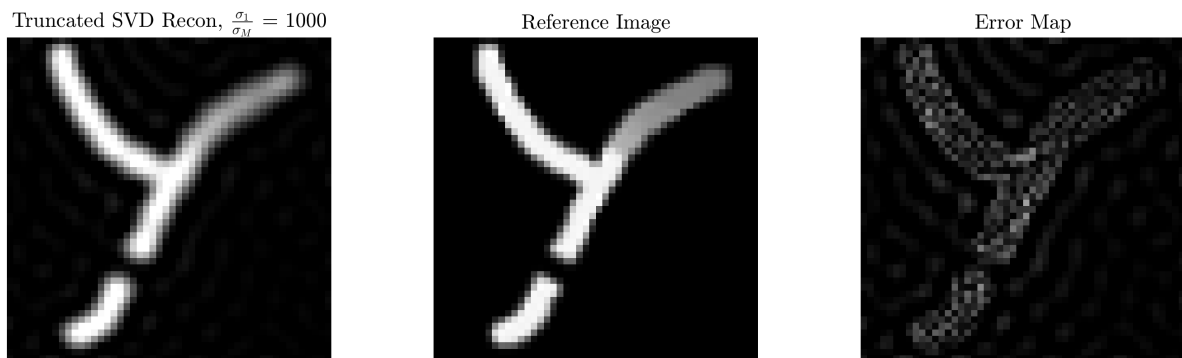


Figure 2.4.5: Reconstruction with Thresholding for Condition Number of 10^3

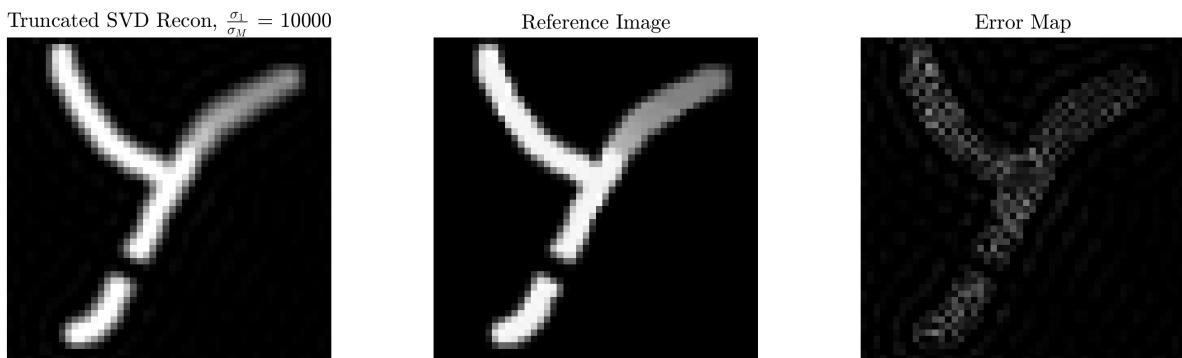


Figure 2.4.6: Reconstruction with Thresholding for Condition Number of 10^4

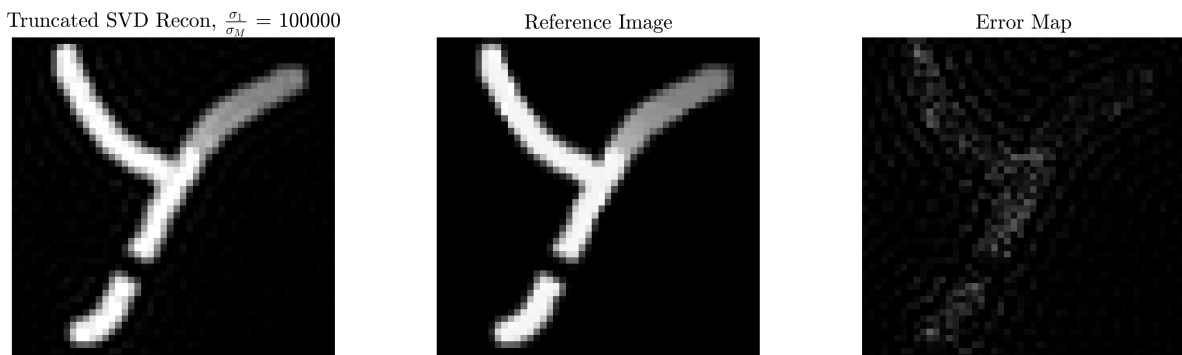


Figure 2.4.7: Reconstruction with Thresholding for Condition Number of 10^5

Truncated SVD Recon, $\frac{\sigma_1}{\sigma_M} = 1000000$ 

Reference Image



Error Map

Figure 2.4.8: Reconstruction with Thresholding for Condition Number of 10^6 Truncated SVD Recon, $\frac{\sigma_1}{\sigma_M} = 10000000$ 

Reference Image



Error Map

Figure 2.4.9: Reconstruction with Thresholding for Condition Number of 10^7

We can see an improvement in image quality as we increase the number of included singular values. However, after a certain point, the image quality starts to degrade again. This is because we are also increasing the condition number of the system matrix, which again leads to the matrix becoming more ill-conditioned. The SSIM and PSNR values for these reconstructions are shown in Table 2.4.1 along with a plot of SSIM and PSNR values for different threshold values in Figure 2.4.10.

cond(S)	1	5	10^1	10^2	10^3	10^4	10^5	10^6	10^7
SSIM	0.0151	0.1774	0.3030	0.4219	0.6660	0.8588	0.8196	0.8859	0.8539
PSNR	11.6012	12.1891	13.6486	19.7741	26.4567	27.9205	30.6093	31.1224	30.1780

Table 2.4.1: SSIM and PSNR Values for Different Threshold Values

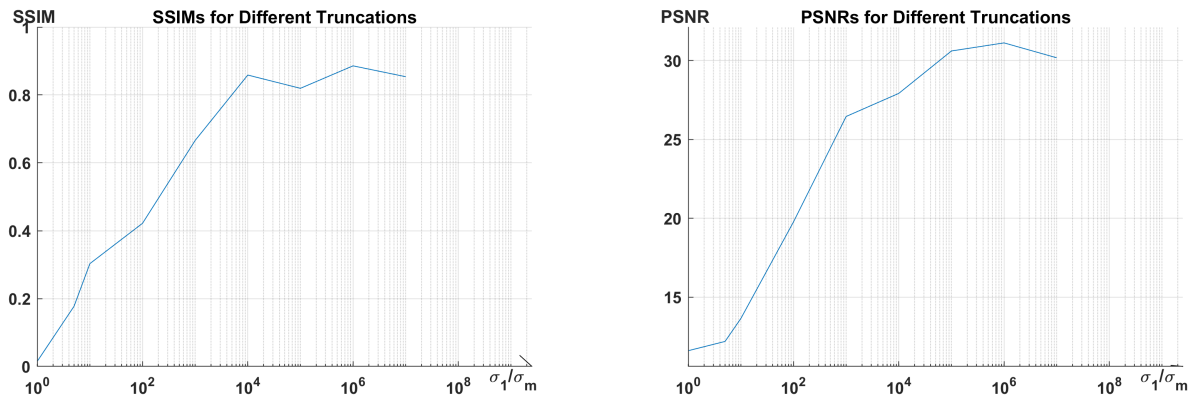


Figure 2.4.10: SSIM and PSNR Values for Different Threshold Values

```

1 cond_nums_desired = [1 5 10 100 10^3 10^4 10^5 10^6 10^7];
2 PSNRs2_4 = zeros(size(cond_nums_desired));
3 SSIMs2_4 = PSNRs2_4;
4 for iteration = 1:length(cond_nums_desired)
5
6     [val,M] = min(abs(s-(s(1)/cond_nums_desired(iteration)))));
7     Sigma_new = Sigma(1:M,1:M);
8     U_new = U(:,1:M);
9     V_new = V(:,1:M);
10
11     Sigma_new_inv = invert_sigma(Sigma_new);
12     c = V_new * Sigma_new_inv * ctranspose(U_new) * u;
13     ima = reshape(c,50,50);
14     ima(ima<0) = 0;
15     plot_comparative_images(ima,m_ref,save_image, strcat("q24",num2str(iteration),"_c_1_"),
        ↳ omp.png"), strcat("Truncated SVD Recon, $\frac{\sigma_1}{\sigma_M}$ = ",
        ↳ num2str(cond_nums_desired(iteration)), "Reference Image",max_val)
16     SSIMs2_4(iteration) = ssim(ima/max_val,m_ref/max_val);
17     PSNRs2_4(iteration) = psnr(ima/max_val,m_ref/max_val);
18 end
19 figure();
20 subplot(1,2,1)
21 finesemilogx(cond_nums_desired,SSIMs2_4,"off","SSIMs for Different
    ↳ Truncations","\sigma_1/\sigma_m","SSIM",[0 9.4],[0 1],"off",[800 400],"","-")
22 subplot(1,2,2)
23 finesemilogx(cond_nums_desired,PSNRs2_4,"off","PSNRs for Different
    ↳ Truncations","\sigma_1/\sigma_m","PSNR",[0 9.4],[min(PSNRs2_4)-1
    ↳ max(PSNRs2_4)+1],"off",[800 400],"","-")
24 set(gcf, 'Units', 'Normalized', 'OuterPosition', [0, 0, 0.8, 0.4]);
25 if save_image
26     exportgraphics(gcf, strcat("q24_ssimsnr.png"), 'Resolution', 300)
27 end

```

Question 2.5

We can also filter the singular values using a regularization parameter instead of a hard threshold. We do this by filtering the singular values as:

$$s'_i = \frac{s_i^2 + \lambda}{s_i} \quad (2.5.1)$$

Another method of regularization adds a regularization term to the least squares problem. This can be implemented as follows:

$$(S^H S + \lambda I)c = S^H u \quad (2.5.2)$$

We are asked to show that these two methods are indeed equivalent. We can start by rearranging the second equation and substituting the SVD of the system matrix as follows:

$$c = (S^H S + \lambda I)^{-1} S^H u \quad (2.5.3)$$

$$c = (V \Sigma^H U^H U \Sigma V^H + \lambda I)^{-1} V \Sigma^H U^H u \quad (2.5.4)$$

Since U and V are unitary matrices, we can simplify this equation as follows:

$$c = (V \Sigma^H \Sigma V^H + \lambda I)^{-1} V \Sigma^H U^H u \quad (2.5.5)$$

$$c = V(\Sigma^H \Sigma + \lambda I)^{-1} \Sigma^H U^H u \quad (2.5.6)$$

Since Σ is a diagonal matrix, we can easily compute the inverse of $\Sigma^H \Sigma + \lambda I$ as follows:

$$(\Sigma^H \Sigma + \lambda I)^{-1} = \text{diag}\left(\frac{1}{s_i^2 + \lambda}\right) \quad (2.5.7)$$

We can then substitute this back into the equation for c to obtain:

$$c = V \text{diag} \left(\frac{1}{s_i^2 + \lambda} \right) \Sigma^H U^H u \quad (2.5.8)$$

We can then rearrange this equation to obtain:

$$c = V \text{diag} \left(\frac{s_i}{s_i^2 + \lambda} \right) U^H u \quad (2.5.9)$$

Thus we have shown that solving the regularized least squares problem is equivalent to filtering the singular values as follows:

$$\tilde{\Sigma}^{-1} = \text{diag} \left(\frac{s_i}{s_i^2 + \lambda} \right) \quad (2.5.10)$$

This shows that the two methods of regularization are indeed equivalent, as they both result in the same filtered singular values.

Question 2.6

We can filter the singular values using a regularization parameter λ . By choosing $\lambda = \sigma_1 \sigma_N$, we ensure that $\lambda \ll \sigma_{\max}^2$ and $\lambda \gg \sigma_{\min}^2$. The filtered singular values are shown in Figure 2.6.1.

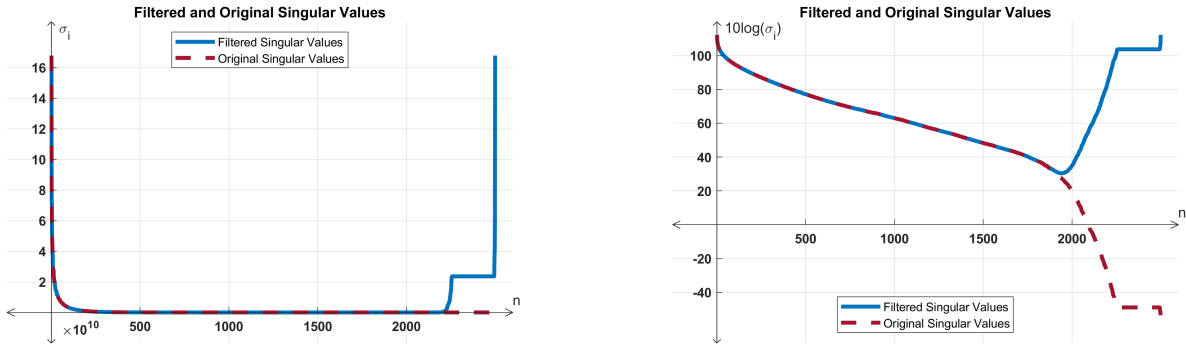


Figure 2.6.1: Filtered Singular Values with Regularization Parameter $\lambda = \sigma_1 \sigma_N$

We can then reconstruct the image using these filtered singular values. The resulting reconstruction is shown in Figure 2.6.2.

Filtered SVD Recon, $\lambda = 3.024 \times 10^5$



Reference Image



Error Map

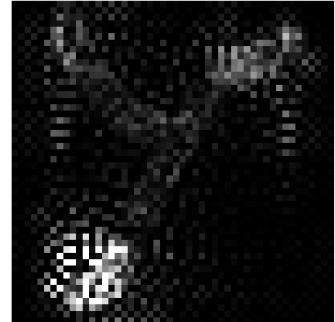


Figure 2.6.2: Reconstruction with Regularization Parameter $\lambda = \sigma_1 \sigma_N$

$$\text{SSIM} = 0.5665, \text{PSNR} = 19.2721$$

```

1 lambda = Sigma(1,1) * Sigma(end,end)
2 s_reg = (s.^2 + lambda)./s;
3 Sigma_reg = diag(s_reg);
4 Sigma_reg_inv = invert_sigma(Sigma_reg);
5 c = V * Sigma_reg_inv * ctranspose(U) * u;
6 ima = reshape(c,50,50);
7 ima(ima<0) = 0;
8 figure;
9 subplot(1,2,1)
10 p1 = fineplot(1:numel(s_reg),s_reg,"Filtered and Original Singular
    ↳ Values","n","\sigma_i",[-250 2600],[-2 19]*(10^10),"on",[1700 400],"Filtered
    ↳ Singular Values","--");
11 p2 = fineplot(1:numel(s),s,"Filtered and Original Singular
    ↳ Values","n","\sigma_i",[-250 2600],[-2 19]*(10^10),"on",[1700 400],"Original
    ↳ Singular Values","--");
12 p1.LineWidth = 3;
13 p2.LineWidth = 3;
14 legend("Location","north")
15 subplot(1,2,2)
16 p1 = fineplot(1:numel(s_reg),10*log10(s_reg),"Filtered and Original Singular
    ↳ Values","n","\sigma_i",[-250 2600],[-70 120],"on",[1700 400],"Filtered Singular
    ↳ Values","--");
17 p2 = fineplot(1:numel(s),10*log10(s),"Filtered and Original Singular
    ↳ Values","n","10log(\sigma_i",[-250 2600],[-70 120],"on",[1700 400],"Original
    ↳ Singular Values","--");
18 p1.LineWidth = 3;
19 p2.LineWidth = 3;
20 legend("Location","south")
21 set(gcf, 'Units', 'Normalized', 'OuterPosition', [0, 0, 0.9, 0.45]);
22 if save_image
23     exportgraphics(gcf, strcat("q26_sigma.png"), 'Resolution', 300)
24 end
25 plot_comparetive_images(ima,m_ref,save_image,"q26_comp.png",strcat("Filtered SVD
    ↳ Recon, $\lambda$ = ",num2sci(lambda,"latex")), "Reference Image",max_val)
26 PSNR2_6 = psnr(ima/max_val,m_ref/max_val);
27 SSIM2_6 = ssim(ima/max_val,m_ref/max_val);

```

Question 2.7

We can experiment with different values of λ to see how it affects the reconstruction. We will sweep different values and pick the one that gives us the best SSIM and PSNR values which are visible in Figure 2.7.1.

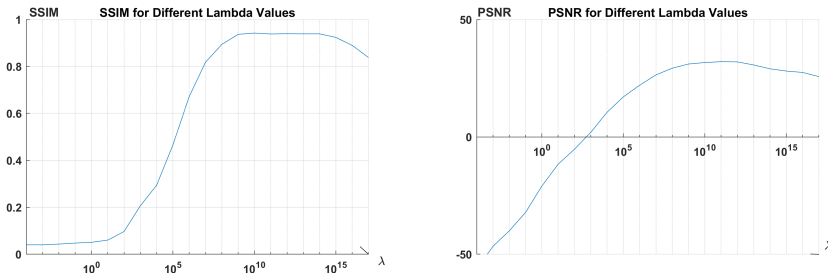


Figure 2.7.1: SSIM and PSNR Values for Different Regularization Parameters λ

We can select $\lambda^* = 4.171 \times 10^9$. We will then reconstruct the image using three different regularization parameters, which are λ^* , $\lambda^*/10$, and $10\lambda^*$. The resulting reconstructions are shown in Figures 2.7.2 through 2.7.4.

Filtered SVD Recon, $\lambda = 4.171 \times 10^8$



Reference Image



Error Map



Figure 2.7.2: Reconstruction with Regularization Parameter $\lambda^* = 4.171 \times 10^8$

Filtered SVD Recon, $\lambda = 4.171 \times 10^9$



Reference Image



Error Map



Figure 2.7.3: Reconstruction with Regularization Parameter $\lambda^*/10 = 4.71 \times 10^9$

Filtered SVD Recon, $\lambda = 4.171 \times 10^{10}$



Reference Image



Error Map



Figure 2.7.4: Reconstruction with Regularization Parameter $\lambda^* \times 10 = 4.171 \times 10^{10}$

Regularization Parameter λ	$\lambda^*/10$	λ^*	$\lambda^* \times 10$
SSIM	0.9257	0.9423	0.9398
PSNR	30.5603	31.5024	31.9216

Table 2.7.1: SSIM and PSNR Values for Different Regularization Parameters λ

```

1  lambda_sweep = logspace(-4,17,22);
2  PSNRs_Sweep = zeros(size(lambda_sweep));
3  SSIMs_Sweep = PSNRs_Sweep;
4  for iteration = 1:numel(lambda_sweep)
5      s_reg = (s.^2 + lambda_sweep(iteration))./s;
6      Sigma_reg = diag(s_reg);
7      Sigma_reg_inv = invert_sigma(Sigma_reg);
8      c = V * Sigma_reg_inv * ctranspose(U) * u;
9      ima = reshape(c,50,50);
10     ima(ima<0) = 0;
11     PSNRs_Sweep(iteration) = psnr(ima/max_val,m_ref/max_val);
12     SSIMs_Sweep(iteration) = ssim(ima/max_val,m_ref/max_val);
13 end
14 figure;
15 subplot(1,2,1)
16 finesemilogx(lambda_sweep,SSIMs_Sweep,"off","SSIM for Different Lambda
↪ Values","\lambda","SSIM",[-4 17],[0 1],"off",[800 400],"","-");
17 subplot(1,2,2)
18 finesemilogx(lambda_sweep,PSNRs_Sweep,"off","PSNR for Different Lambda
↪ Values","\lambda","PSNR",[-4 17],[-50 50],"off",[800 400],"","-");
19 set(gcf,'position',[0, 0, 800, 400]);
20 set(gcf,'Units','Normalized','OuterPosition',[0, 0.04, 0.8, 0.4]);
21 if save_image
22     exportgraphics(gcf,strcmp("q27_ssimpsnr.png"),'Resolution',300)
23 end
24 lambda = 4.171 * (10^9);
25 PSNRs2_7 = zeros(1,3);
26 SSIMs2_7 = zeros(1,3);
27 lambdas2 = [lambda/10 lambda lambda*10];
28 for iteration = 1:3
29     s_reg = (s.^2 + lambdas2(iteration))./s;
30     Sigma_reg = diag(s_reg);
31     Sigma_reg_inv = invert_sigma(Sigma_reg);
32     c = V * Sigma_reg_inv * ctranspose(U) * u;
33     ima = reshape(c,50,50);
34     ima(ima<0) = 0;
35     PSNRs2_7(iteration) = psnr(ima/max_val,m_ref/max_val);
36     SSIMs2_7(iteration) = ssim(ima/max_val,m_ref/max_val);
37     plot_comparative_images(ima,m_ref,save_image,strcmp("q27_",num2str(iteration),"_
↪ comp.png"),strcmp("Filtered SVD Recon, $\lambda$ =
↪ ",num2sci(lambdas2(iteration),"latex")), "Reference Image",max_val)
38 end

```

Question 2.8

We used PSNR and SSIM values to select the optimal regularization parameter λ^* . However, we may not always have access to the reference image to calculate these metrics. In such cases, we can use other methods to select the regularization parameter. One common method is the L-curve method, which plots the solution norm against the residual norm for different values of λ . The optimal λ is then selected as the point on the curve where the curvature is maximized. We can generate the L-curve for our problem and select the optimal λ using this method. The resulting L-curve is shown in Figure 2.8.1.

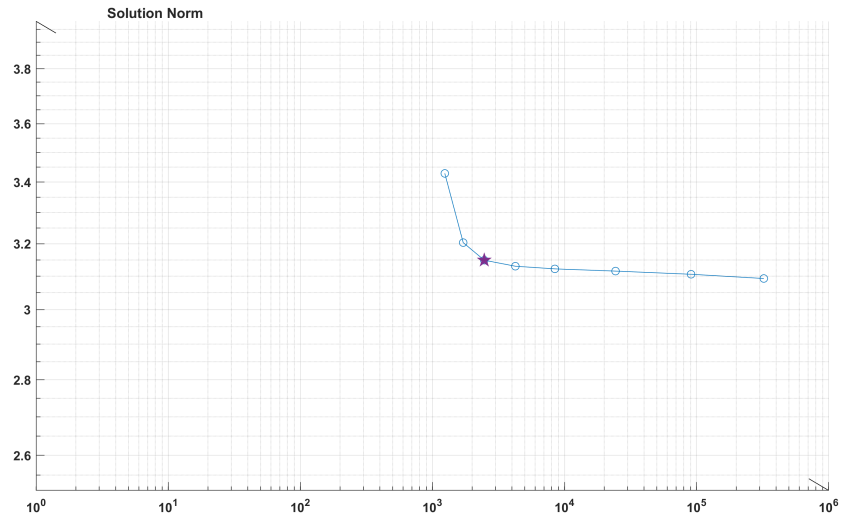


Figure 2.8.1: L-curve for Selecting Regularization Parameter λ

The selected λ value using the L-curve method is $\lambda_L = 3.024 \times 10^7$. We can then use this λ value to reconstruct the image. The resulting reconstruction is shown in Figure 2.8.2.

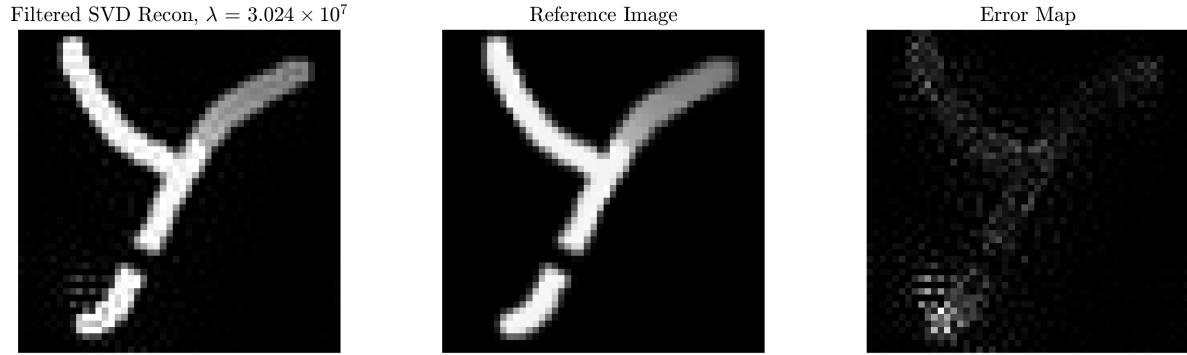


Figure 2.8.2: Reconstruction with Regularization Parameter $\lambda_L = 3.024 \times 10^7$

$$\text{SSIM} = 0.8596, \text{PSNR} = 27.9332$$


```

1 residual_norms = [];
2 solution_norms = [];
3 lambda_sweep2 = (s(1)*s(end))*logspace(0,7,8);
4 for lambda = lambda_sweep2
5     s_reg = (s.^2 + lambda)./s;
6     Sigma_reg = diag(s_reg);
7     Sigma_reg_inv = invert_sigma(Sigma_reg);
8     c = V * Sigma_reg_inv * ctranspose(U) * u;
9     ima = reshape(c,50,50);
10    ima(ima<0) = 0;
11    residual_norm = norm(S*c - u);
12    solution_norm = norm(c);
13    residual_norms = [residual_norms residual_norm];
14    solution_norms = [solution_norms solution_norm];
15 end
16 lambda = lambda_sweep2(3)
17 figure;
18 finelogslog(residual_norms, solution_norms, "off","L-Curve","Residual Norm","Solution
↪ Norm",[0 6],[0.4 0.6],"on",[400 400],"","-o");
19 plot(residual_norms(3),solution_norms(3),"p","LineWidth",2)
20 grid on
21 set(gcf, 'Units', 'Normalized', 'OuterPosition', [0, 0.04, 0.7, 0.7]);
22 if save_image
23     exportgraphics(gcf, strcat("q28_lcurve.png"), 'Resolution', 300)
24 end
25 s_reg = (s.^2 + lambda)./s;
26 Sigma_reg = diag(s_reg);
27 Sigma_reg_inv = invert_sigma(Sigma_reg);
28 c = V * Sigma_reg_inv * ctranspose(U) * u;
29
30 ima = reshape(c,50,50);
31 ima(ima<0) = 0;
32 PSNR2_8 = psnr(ima/max_val,m_ref/max_val);
33 SSIM2_8 = ssim(ima/max_val,m_ref/max_val);
34 plot_comparative_images(ima,m_ref,save_image, strcat("q28_comp.png"), strcat("Filtered
↪ SVD Recon, $\lambda$ = ",num2sci(lambda,"latex")), "Reference Image", max_val)

```

Question 3

In this question, we will implement the Kachmarz method for image reconstruction. This method is an iterative algorithm that updates the reconstruction by projecting the current estimate onto the hyperplanes defined by the rows of the system matrix and the corresponding measurements. We will implement different methods such as norm thresholding and regularization to improve the reconstruction quality. We will analyse the effects of these methods on the reconstruction quality by looking at the PSNR and SSIM values along with visual comparisons of the reconstructions and error maps.

Question 3.1

We will first threshold the rows of the system matrix based on their norms. A graph that shows norms of each row of the matrix is given in Figure 3.1.1.

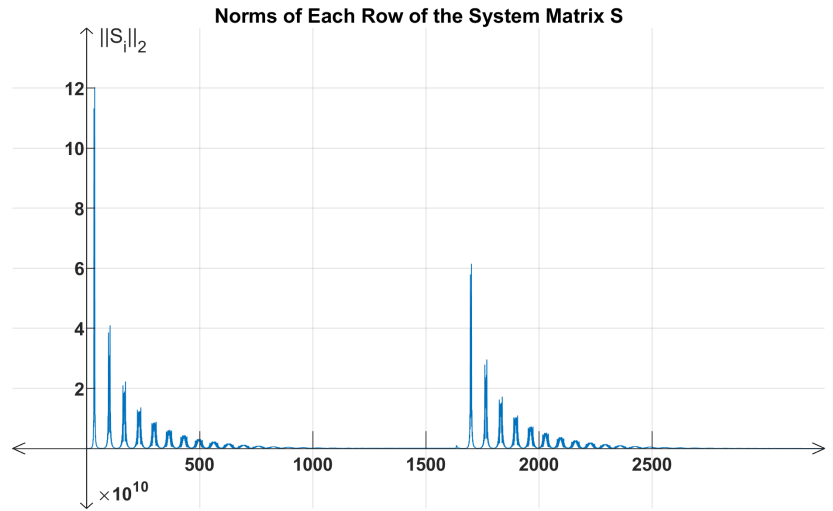


Figure 3.1.1: Norms of Each Row of the System Matrix

We will choose a threshold value of $max_norm/10$ and omit the rows that have norms below this threshold. We will also omit the corresponding measurements for these rows. The sizes of the matrix and measurement vector after thresholding are 44×2500 and 44×1 respectively. This means that we are only using a small portion of the available data for reconstruction, which may lead to significant information losses.

```

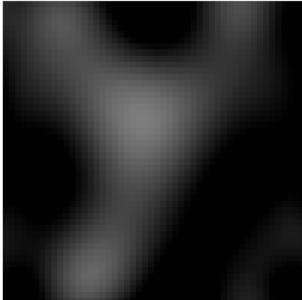
1  S = load("S.mat");
2  S = S.S;
3  size_S = size(S);
4  norms = sqrt(sum(S.^2,2));
5  clf;
6  fineplot(1:numel(norms),norms,"Norms of Each Row of the System Matrix
   ↪ S","i","||S_i||_2",[-0.1*size_S(1) size_S(1)],[-2*10^10 14*10^10],"off",[400 400],"","-");
7  set(gcf, 'Units', 'Normalized', 'OuterPosition', [0, 0.04, 0.5, 0.5]);
8  if save_image
9      exportgraphics(gcf,strcat("q31_norms.png"),'Resolution',300)
10 end
11 max_norm = max(norms);
12 thresh1 = max_norm/10;
13 S_thresh = S(norms>thresh1,:);
14 u_thresh = u(norms>thresh1);

```

Question 3.2

We can use these thresholded system matrix and measurement vector to reconstruct the image using the Kaczmarz method. The resulting reconstructions for each iteration are shown in Figure 3.2.1 through 3.2.10.

Kaczmarz Recon, Threshold = $\frac{max_norm}{10}$, Iteration = 1



Reference Image



Error Map

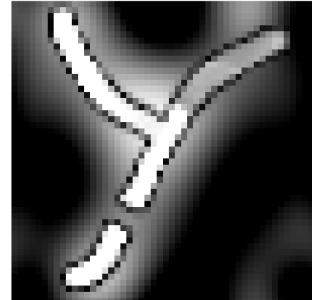
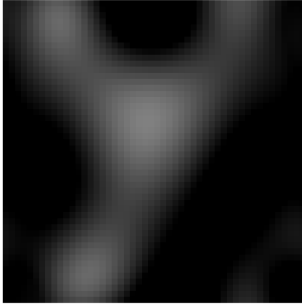
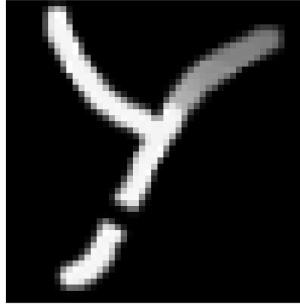


Figure 3.2.1: Reconstruction After 1 Iteration

Kaczmarz Recon, Threshold = $\frac{max_norm}{10}$, Iteration = 2



Reference Image



Error Map

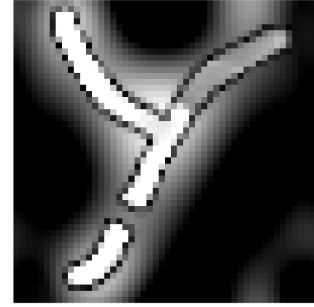
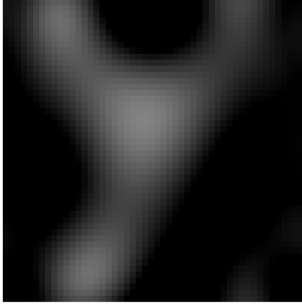
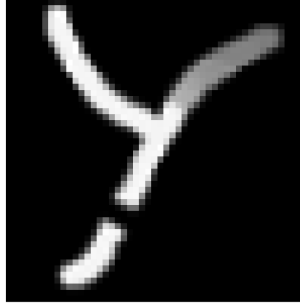


Figure 3.2.2: Reconstruction After 2 Iterations

Kaczmarz Recon, Threshold = $\frac{max_norm}{10}$, Iteration = 3



Reference Image

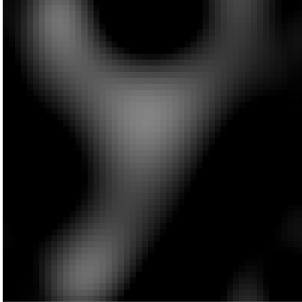


Error Map



Figure 3.2.3: Reconstruction After 3 Iterations

Kaczmarz Recon, Threshold = $\frac{max_norm}{10}$, Iteration = 4



Reference Image



Error Map

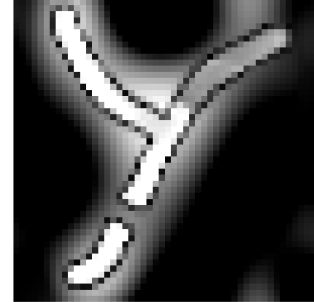
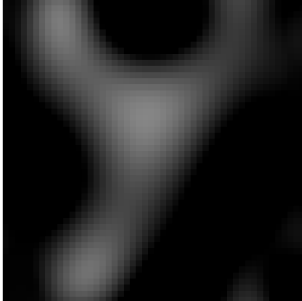


Figure 3.2.4: Reconstruction After 4 Iterations

Kaczmarz Recon, Threshold = $\frac{max_norm}{10}$, Iteration = 5



Reference Image



Error Map

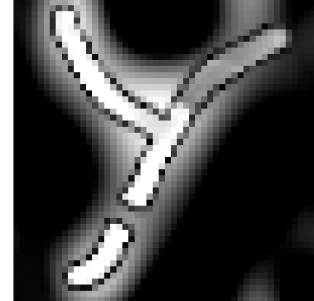
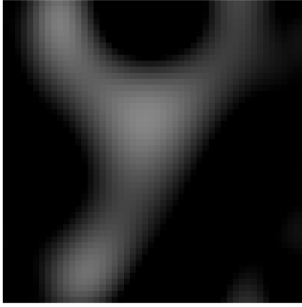
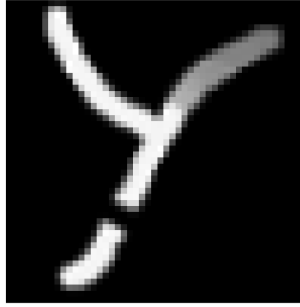


Figure 3.2.5: Reconstruction After 5 Iterations

Kaczmarz Recon, Threshold = $\frac{max_norm}{10}$, Iteration = 6



Reference Image



Error Map

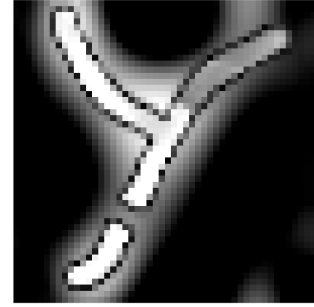
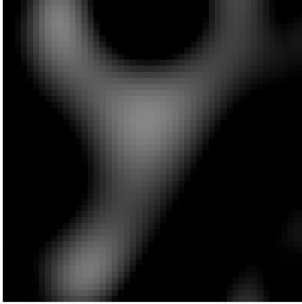
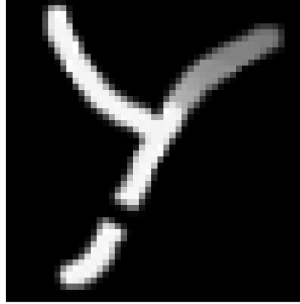


Figure 3.2.6: Reconstruction After 6 Iterations

Kaczmarz Recon, Threshold = $\frac{max_norm}{10}$, Iteration = 7



Reference Image



Error Map

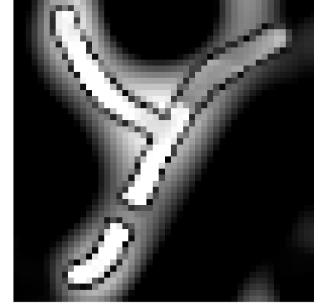
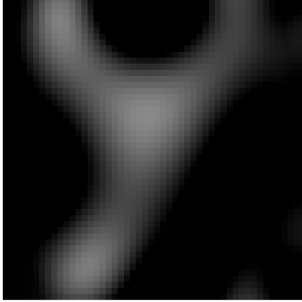
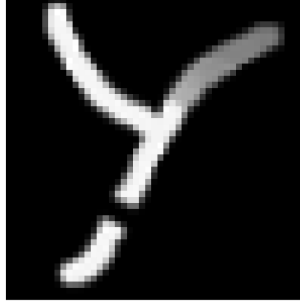


Figure 3.2.7: Reconstruction After 7 Iterations

Kaczmarz Recon, Threshold = $\frac{max_norm}{10}$, Iteration = 8



Reference Image



Error Map

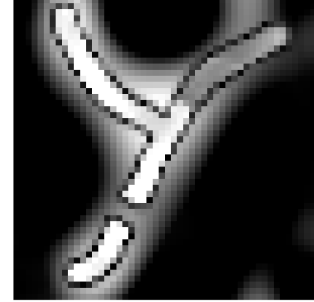
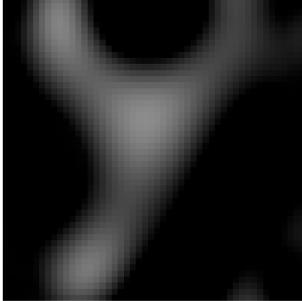
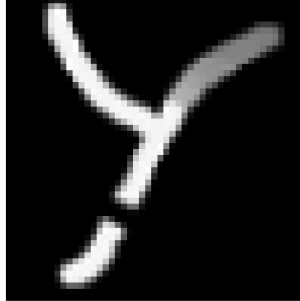


Figure 3.2.8: Reconstruction After 8 Iterations

Kaczmarz Recon, Threshold = $\frac{max_norm}{10}$, Iteration = 9



Reference Image



Error Map

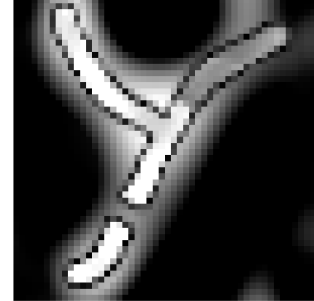
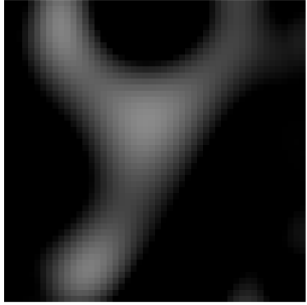


Figure 3.2.9: Reconstruction After 9 Iterations

Kaczmarz Recon, Threshold = $\frac{max_norm}{10}$, Iteration = 10



Reference Image



Error Map



Figure 3.2.10: Reconstruction After 10 Iterations

We can see that the reconstruction quality improves with each iteration, but after a certain point, the improvement slows down. We also fail to reconstruct the image accurately, as the threshold is too high and removes significant information. This is also observable in the PSNR and SSIM values, which are shown in Figure 3.2.11.

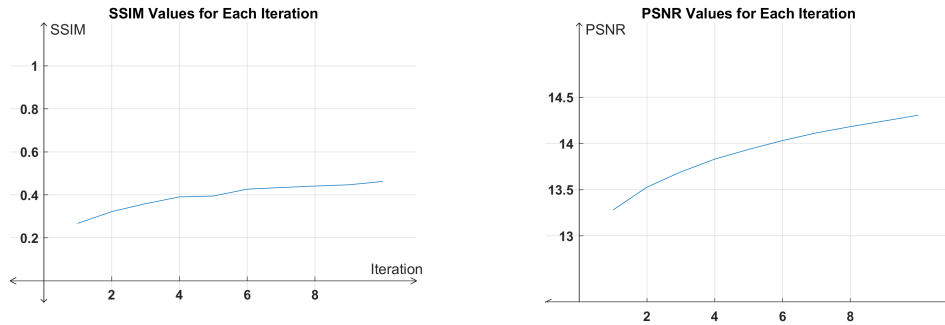


Figure 3.2.11: SSIM and PSNR Values for Each Iteration of the Kaczmarz Method

```
1 rng(67); %prevents differences between executions
2 c = kaczmarz_method(S_thresh,u_thresh,50,0,{1,save_image,max_val,m_ref,"$\frac{max\_norm}{10}$",[0 max_val]});
```

Question 3.3

We can use different threshold values to see how it affects the reconstruction quality. We can experiment with 4 different threshold values, $max_norm \times 10^{-4}$, $max_norm \times 10^{-3}$, $max_norm \times 10^{-2}$, and $max_norm \times 10^{-1}$. The resulting reconstructions for each threshold value are shown in Figures 3.3.1 through 3.3.4.

Kaczmarz Recon, Threshold = $\frac{max_norm}{10000}$



Reference Image



Error Map



Figure 3.3.1: Reconstruction with Threshold Value of $max_norm \times 10^{-4}$

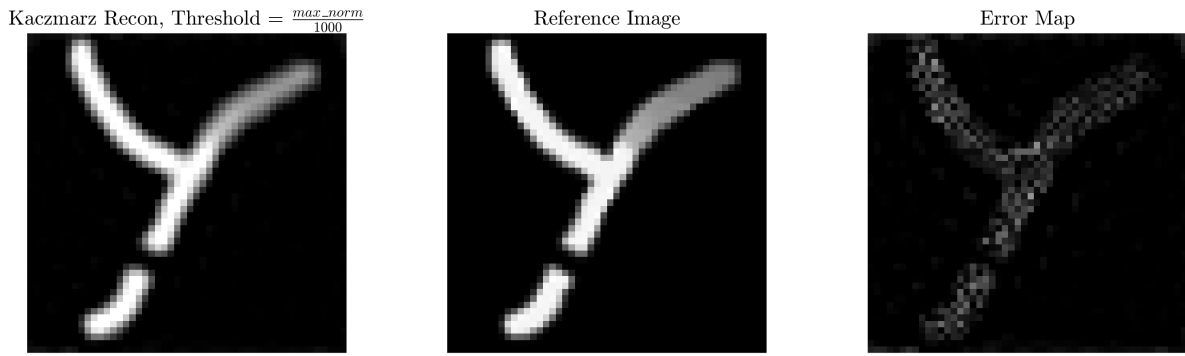


Figure 3.3.2: Reconstruction with Threshold Value of $max_norm \times 10^{-3}$

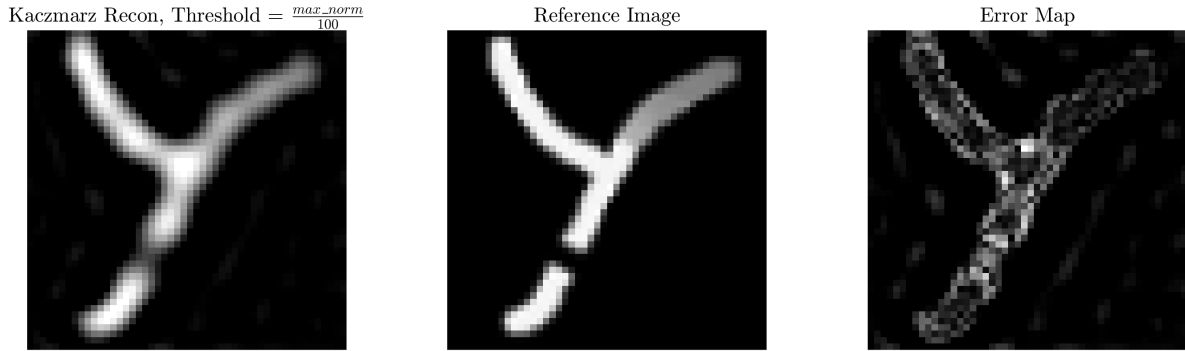


Figure 3.3.3: Reconstruction with Threshold Value of $max_norm \times 10^{-2}$

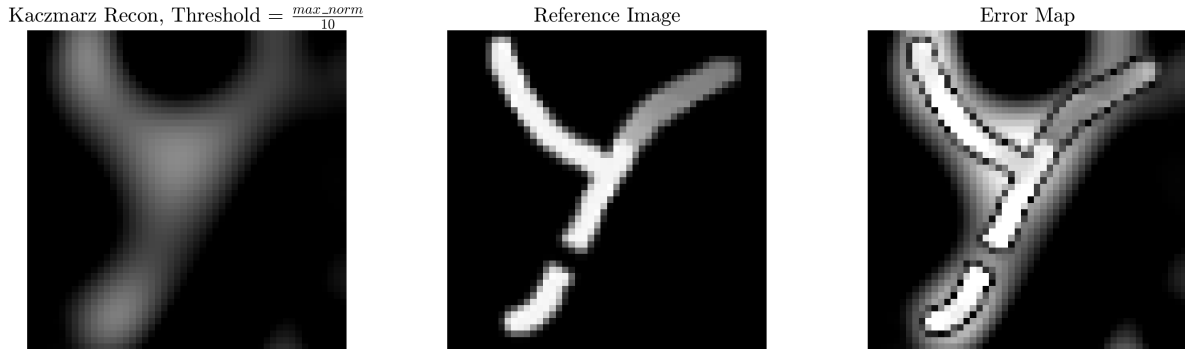


Figure 3.3.4: Reconstruction with Threshold Value of $max_norm \times 10^{-1}$

We can see that as we include more rows in the system matrix, the reconstruction quality improves. As we are working with a noise-free system, we can include more rows without worrying about the noise amplification. The main trade-off here is the computational cost and numerical errors due to ill-conditioning against information.

Threshold Value	$max_norm \times 10^{-4}$	$max_norm \times 10^{-3}$	$max_norm \times 10^{-2}$	$max_norm \times 10^{-1}$
SSIM	0.9371	0.9111	0.7408	0.4622
PSNR	28.9215	27.9475	22.9577	14.3062

Table 3.3.1: SSIM and PSNR Values for Different Threshold Values

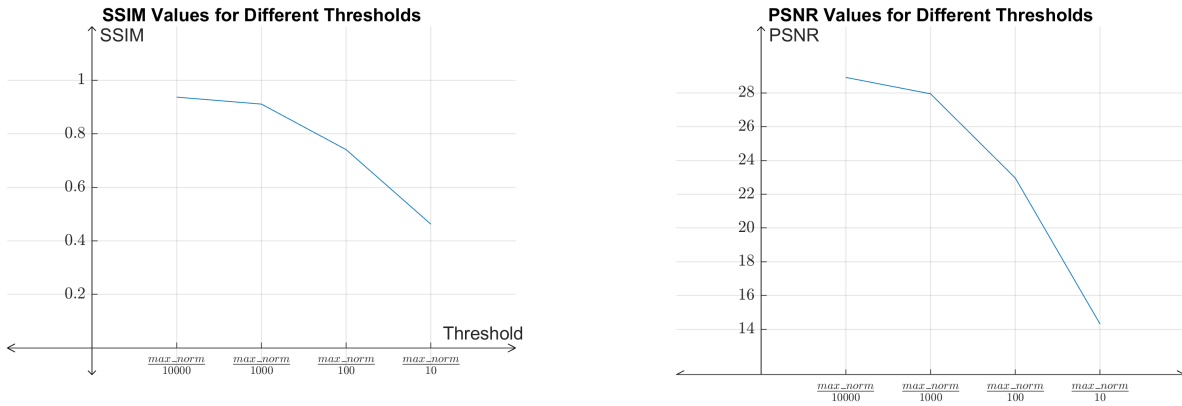


Figure 3.3.5: SSIM and PSNR Values for Different Threshold Values

```

1  threshs = [(10^-4) (10^-3) (10^-2) (10^-1)];
2  PSNRs3_3 = zeros(size(threshs));
3  SSIMs3_3 = zeros(size(threshs));
4  for iter = 1:numel(threshs)
5      S_thresh_loop = S(norms>threshs(iter)*max_norm,:);
6      u_thresh_loop = u(norms>threshs(iter)*max_norm);
7      rng(67)
8      c = kaczmarz_method(S_thresh_loop,u_thresh_loop,50,0,{0,0,0,0,0,0});
9      ima = reshape(c,50,50);
10     ima(ima<0) = 0;
11     plot_comparative_images(ima,m_ref,save_image,sprintf("Q33_comp_%i",iter),sprintf
        ↳ ("Kaczmarz Recon, Threshold =
        ↳ '%s",strcat("$\frac{max\_norm}{10000}$",num2str(1/threshs(iter))," } $")), "Reference
        ↳ Image",max_val)
12     PSNRs3_3(iter) = psnr(ima,m_ref);
13     SSIMs3_3(iter) = ssim(ima,m_ref);
14 end
15 figure();
16 subplot(1,2,1)
17 fineplot(1:numel(threshs),SSIMs3_3,"SSIM Values for Different
        ↳ Thresholds","Threshold","SSIM",[-1 5],[-0.1 1.2],"off",[800 400],"","-");
18 set(gca,"TickLabelInterpreter",'latex')
19 xticklabels({"","$\frac{max\_norm}{10000}$","$\frac{max\_norm}{1000}$","$\frac{max\_norm}{100}$",
        ↳ "$\frac{max\_norm}{10}$"})
20 subplot(1,2,2)
21 fineplot(1:numel(threshs),PSNRs3_3,"PSNR Values for Different
        ↳ Thresholds","Threshold","PSNR",[-1 5],[min(PSNRs3_3)-3
        ↳ max(PSNRs3_3)+3],"off",[800 400],"","-");
22 set(gca,"TickLabelInterpreter",'latex')
23 xticklabels({"","$\frac{max\_norm}{10000}$","$\frac{max\_norm}{1000}$","$\frac{max\_norm}{100}$",
        ↳ "$\frac{max\_norm}{10}$"})
24 set(gcf,'Units','Normalized','OuterPosition',[0,0.04,0.8,0.4]);
25 if save_image
26     exportgraphics(gcf,strcat("q33_ssimsnr.png"),'Resolution',300)
27 end

```

Question 3.4

We can use regularization with Kaczmarz method as well to improve the reconstruction quality. We will again use $\lambda = \sigma_1 \sigma_N$ as the regularization parameter. The resulting reconstruction is shown in Figure 3.4.1 through 3.4.10.

Kaczmarz Recon, $\lambda = 3.024 \times 10^5$, Iteration = 1



Reference Image



Error Map

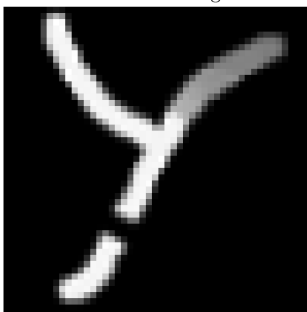


Figure 3.4.1: Reconstruction with Regularization After 1 Iteration

Kaczmarz Recon, $\lambda = 3.024 \times 10^5$, Iteration = 2



Reference Image



Error Map



Figure 3.4.2: Reconstruction with Regularization After 2 Iterations

Kaczmarz Recon, $\lambda = 3.024 \times 10^5$, Iteration = 3



Reference Image



Error Map



Figure 3.4.3: Reconstruction with Regularization After 3 Iterations

Kaczmarz Recon, $\lambda = 3.024 \times 10^5$, Iteration = 4



Reference Image



Error Map



Figure 3.4.4: Reconstruction with Regularization After 4 Iterations

Kaczmarz Recon, $\lambda = 3.024 \times 10^5$, Iteration = 5



Reference Image



Error Map

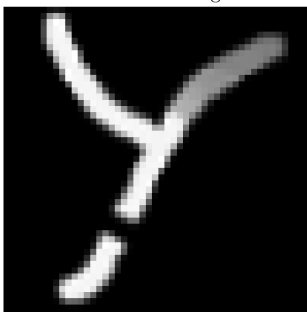


Figure 3.4.5: Reconstruction with Regularization After 5 Iterations

Kaczmarz Recon, $\lambda = 3.024 \times 10^5$, Iteration = 6



Reference Image



Error Map



Figure 3.4.6: Reconstruction with Regularization After 6 Iterations

Kaczmarz Recon, $\lambda = 3.024 \times 10^5$, Iteration = 7



Reference Image



Error Map



Figure 3.4.7: Reconstruction with Regularization After 7 Iterations

Kaczmarz Recon, $\lambda = 3.024 \times 10^5$, Iteration = 8



Reference Image



Error Map



Figure 3.4.8: Reconstruction with Regularization After 8 Iterations

Kaczmarz Recon, $\lambda = 3.024 \times 10^5$, Iteration = 9

Reference Image



Error Map



Figure 3.4.9: Reconstruction with Regularization After 9 Iterations

Kaczmarz Recon, $\lambda = 3.024 \times 10^5$, Iteration = 10

Reference Image



Error Map



Figure 3.4.10: Reconstruction with Regularization After 10 Iterations

The image quality improves with each iteration but there does not seem to be significant improvement after a certain point. This is also observable in the PSNR and SSIM values, which are shown in Figure 3.4.11.

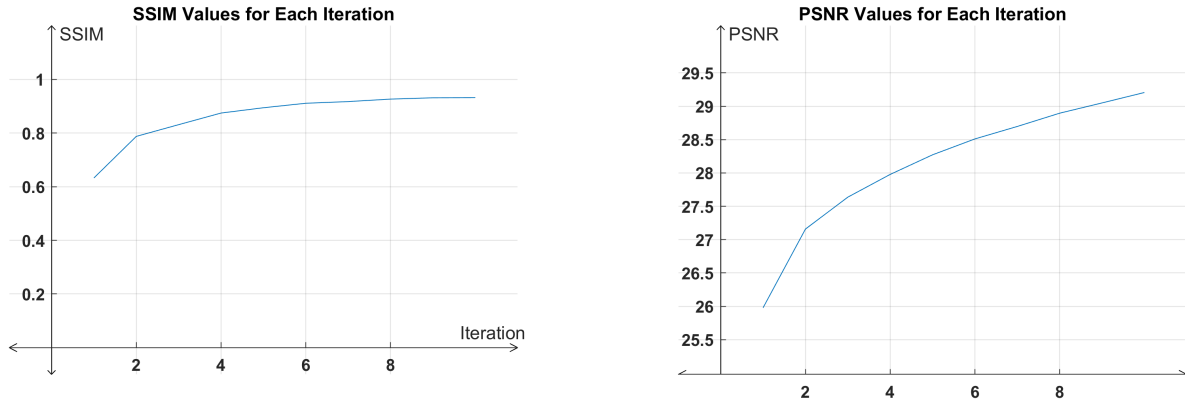


Figure 3.4.11: SSIM and PSNR Values for Each Iteration of the Kachmarz Method with Regularization

```

1  rng(67); %prevents differences between executions
2  lambda = Sigma(1,1) * Sigma(end,end);
3  size_S = size(S);
4  S_reg = [S eye(size_S(1)).*sqrt(lambda)];
5  u_reg = [u ; zeros(size_S(1),1)];
6  c = kaczmarz_method(S_reg,u_reg,50,lambda,{1,save_image,max_val,m_ref,"$\frac{\max\_
   \hookrightarrow$ norm}{10}$",[0 max_val]});

```

Question 3.5

We can choose different regularization parameters. We will reconstruct the image using 5 different lambda values. These lambda values are $\sigma_1^2 \times 10^{-6}$, $\sigma_1^2 \times 10^{-5}$, $\sigma_1^2 \times 10^{-4}$, $\sigma_1^2 \times 10^{-3}$, and $\sigma_1^2 \times 10^{-2}$. The resulting reconstructions for each lambda value are shown in Figures 3.5.1 through 3.5.5.

Regularized Kaczmarz Recon, $\lambda = \sigma_1^2 10^{-6}$



Reference Image



Error Map



Figure 3.5.1: Reconstruction with Regularization Parameter $\lambda = \sigma_1^2 \times 10^{-6}$

Regularized Kaczmarz Recon, $\lambda = \sigma_1^2 10^{-5}$



Reference Image



Error Map

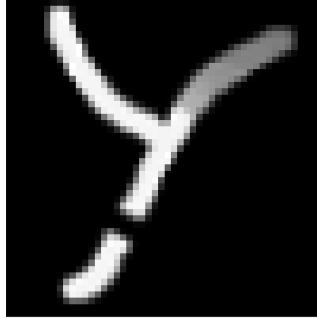


Figure 3.5.2: Reconstruction with Regularization Parameter $\lambda = \sigma_1^2 \times 10^{-5}$

Regularized Kaczmarz Recon, $\lambda = \sigma_1^2 10^{-4}$



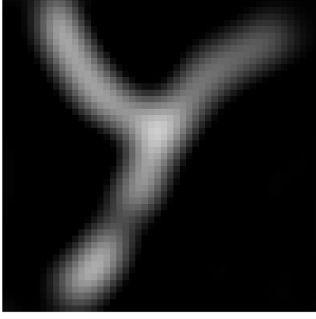
Reference Image



Error Map



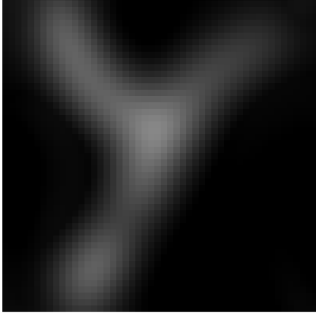
Figure 3.5.3: Reconstruction with Regularization Parameter $\lambda = \sigma_1^2 \times 10^{-4}$

Regularized Kaczmarz Recon, $\lambda = \sigma_1^2 10^{-3}$ 

Reference Image



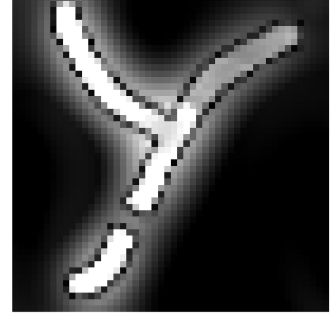
Error Map

Figure 3.5.4: Reconstruction with Regularization Parameter $\lambda = \sigma_1^2 \times 10^{-3}$ Regularized Kaczmarz Recon, $\lambda = \sigma_1^2 10^{-2}$ 

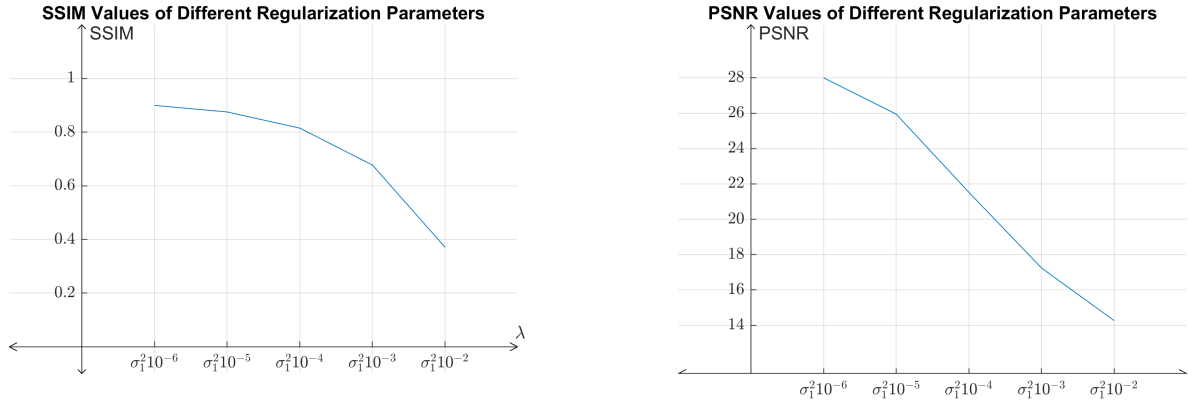
Reference Image



Error Map

Figure 3.5.5: Reconstruction with Regularization Parameter $\lambda = \sigma_1^2 \times 10^{-2}$

We can again see that higher regularization parameters result in worse image quality. The PSNR and SSIM values for each reconstruction is shown in Figure 3.5.6.

Figure 3.5.6: SSIM and PSNR Values for Different Regularization Parameters λ

Lambda Value	$\sigma_1^2 \times 10^{-6}$	$\sigma_1^2 \times 10^{-5}$	$\sigma_1^2 \times 10^{-4}$	$\sigma_1^2 \times 10^{-3}$	$\sigma_1^2 \times 10^{-2}$
SSIM	0.8997	0.8752	0.8150	0.6775	0.3711
PSNR	28.0040	25.9470	21.5237	17.2414	14.2635

Table 3.5.1: SSIM and PSNR Values for Different Regularization Parameters λ

```

1  s1 = Sigma(1,1)^2;
2  lambda_rels = [10^-6 10^-5 10^-4 10^-3 10^-2];
3  PSNRs3_5 = zeros(1,numel(lambda_rels));
4  SSIMs3_5 = zeros(1,numel(lambda_rels));
5  for iter = 1:numel(lambda_rels)
6      S_reg = [S eye(size_S(1)).*sqrt(lambda_rels(iter))];
7      c = kaczmarz_method(S_reg,u_reg,50,lambda_rels(iter)*s1,{0,0,0,0,0});
8      ima = reshape(c,50,50);
9      ima(ima<0) = 0;
10     plot_comparative_images(ima,m_ref,save_image,sprintf("Q35_comp_%i",iter),sprintf
        ↪ ("Regularized Kaczmarz Recon, %s = %s", "\lambda$",strcat("\sigma_1 ^2
        ↪ 10^{",num2str(iter-7)," } $")), "Reference Image",max_val)
11     PSNRs3_5(iter) = psnr(ima/max_val,m_ref/max_val);
12     SSIMs3_5(iter) = ssim(ima/max_val,m_ref/max_val);
13 end
14 figure();
15 subplot(1,2,1)
16 fineplot(1:numel(lambda_rels),SSIMs3_5,"SSIM Values of Different Regularization
        ↪ Parameters", "\lambda", "SSIM", [-1 6], [-0.1 1.2], "off", [800 400], "", "-");
17 set(gca,"TickLabelInterpreter","latex')
18 xticklabels({"", "\sigma_1 ^2 10^{-6}$", "\sigma_1 ^2 10^{-5}$", "\sigma_1 ^2
        ↪ 10^{-4}$", "\sigma_1 ^2 10^{-3}$", "\sigma_1 ^2 10^{-2}$"})
19 subplot(1,2,2)
20 fineplot(1:numel(lambda_rels),PSNRs3_5,"PSNR Values of Different Regularization
        ↪ Parameters", "\lambda", "PSNR", [-1 6], [min(PSNRs3_5)-3 max(PSNRs3_5)+3], "off", [800
        ↪ 400], "", "-");
21 set(gca,"TickLabelInterpreter","latex')
22 xticklabels({"", "\sigma_1 ^2 10^{-6}$", "\sigma_1 ^2 10^{-5}$", "\sigma_1 ^2
        ↪ 10^{-4}$", "\sigma_1 ^2 10^{-3}$", "\sigma_1 ^2 10^{-2}$"})
23 set(gcf, 'Units', 'Normalized', 'OuterPosition', [0, 0.04, 0.8, 0.4]);
24 if save_image
25     exportgraphics(gcf,strcat("q35_ssimsnr.png"), 'Resolution', 300)
26 end

```

Question 3.6

So far we have been working with noise free measurements. To simulate real-life conditions, we will add white gaussian noise to our measurement. The standard deviation of this noise will be equal to $max_norm/1000$. We will then reconstruct the image using thresholded Kaczmarz method using the following threshold values $max_norm \times 10^{-4}$, $max_norm \times 10^{-3}$, $max_norm \times 10^{-2}$, and $max_norm \times 10^{-1}$. The resulting reconstructions for each threshold value are shown in Figures 3.6.1 through 3.6.4.

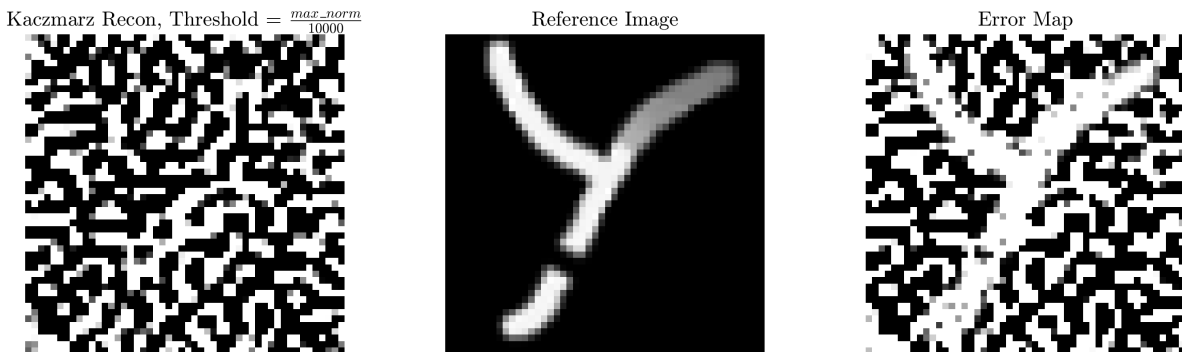


Figure 3.6.1: Reconstruction with Threshold Value of $max_norm \times 10^{-4}$ with Noise

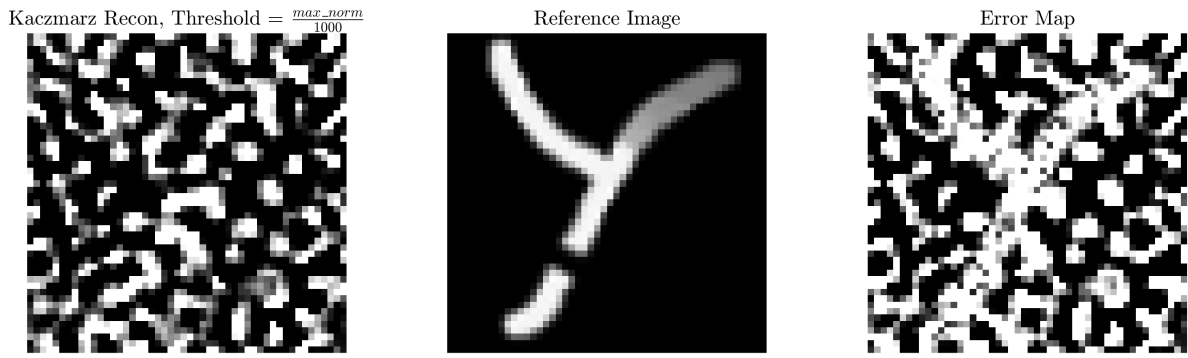


Figure 3.6.2: Reconstruction with Threshold Value of $max_norm \times 10^{-3}$ with Noise

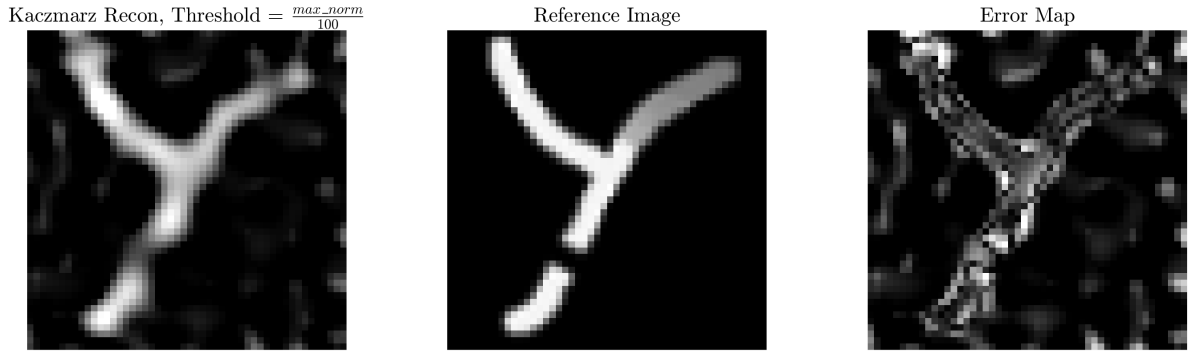


Figure 3.6.3: Reconstruction with Threshold Value of $max_norm \times 10^{-2}$ with Noise

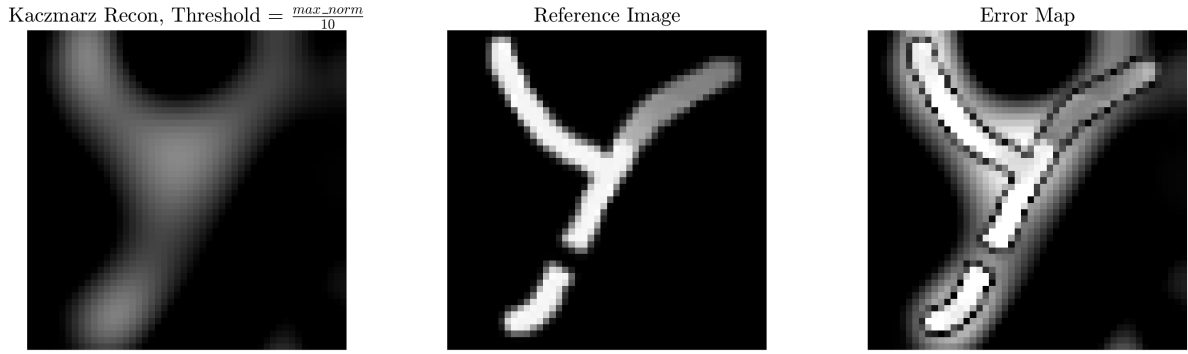


Figure 3.6.4: Reconstruction with Threshold Value of $max_norm \times 10^{-1}$ with Noise

We can see that the reconstructions are significantly worse than the noise-free case. This is because noise is introduced into the system. If the threshold is set too low, the noise is included and the image quality degrades, if the threshold is too high we do not include sufficient information for reconstruction. The tradeoff between information and noise amplification is much more significant in this case. The PSNR and SSIM values are provided in Figure 3.6.5.

Threshold Value	$max_norm \times 10^{-4}$	$max_norm \times 10^{-3}$	$max_norm \times 10^{-2}$	$max_norm \times 10^{-1}$
SSIM	-2.6403e-04	0.0329	0.4305	0.4613
PSNR	-12.2603	1.0494	19.4516	14.2900

Table 3.6.1: SSIM and PSNR Values for Different Threshold Values with Noise

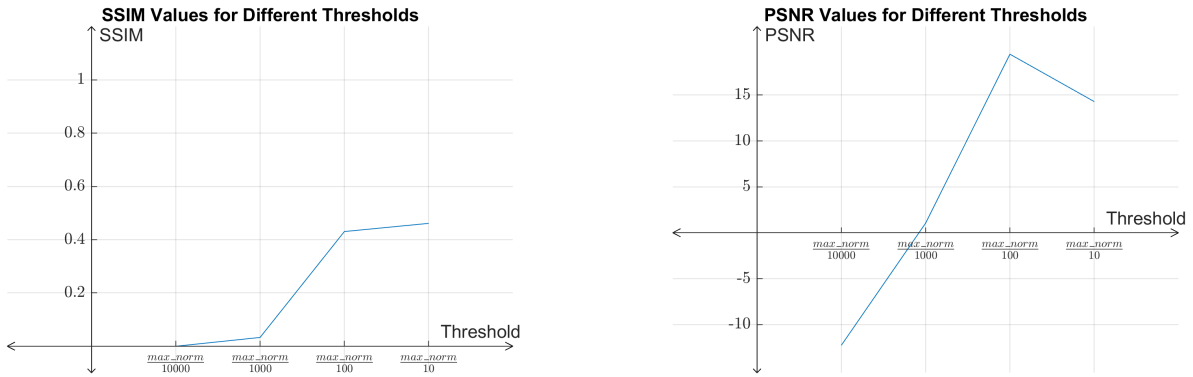


Figure 3.6.5: SSIM and PSNR Values for Different Threshold Values with Noise

```

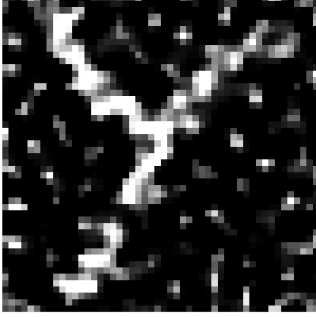
1  rng(67)
2  u_noisy = u + normrnd(0,max_norm/1000,size(u));
3  threshs = [(10^-4) (10^-3) (10^-2) (10^-1)];
4  PSNRs3_6 = zeros(size(threshs));
5  SSIMs3_6 = zeros(size(threshs));
6  for iter = 1:numel(threshs)
7      S_thresh_loop = S(norms>threshs(iter)*max_norm,:);
8      u_thresh_loop = u_noisy(norms>threshs(iter)*max_norm);
9      rng(67)
10     c = kaczmazr_method(S_thresh_loop,u_thresh_loop,50,0,{0,0,0,0,0,0});
11     ima = reshape(c,50,50);
12     ima(ima<0) = 0;
13     plot_comparative_images(ima,m_ref,save_image,sprintf("Q36_comp_%i",iter),sprintf
        ↪ ("Kaczmazr Recon, Threshold =
        ↪ %s",strcat("\frac{max\_norm}{",num2str(1/threshs(iter))," } $")), "Reference
        ↪ Image",max_val)
14     PSNRs3_6(iter) = psnr(ima,m_ref);
15     SSIMs3_6(iter) = ssim(ima,m_ref);
16 end
17 figure();
18 subplot(1,2,1)
19 fineplot(1:numel(threshs),SSIMs3_6,"SSIM Values for Different
        ↪ Thresholds", "Threshold", "SSIM", [-1 5], [-0.1 1.2], "off", [800 400], "", "-");
20 set(gca,"TickLabelInterpreter","latex")
21 xticklabels({"", "\frac{max\_norm}{10000}$", "\frac{max\_norm}{1000}$", "\frac{max\_
        ↪ norm}{100}$", "\frac{max\_norm}{10}$"})
22 subplot(1,2,2)
23 fineplot(1:numel(threshs),PSNRs3_6,"PSNR Values for Different
        ↪ Thresholds", "Threshold", "PSNR", [-1 5], [min(PSNRs3_6)-3
        ↪ max(PSNRs3_6)+3], "off", [800 400], "", "-");
24 set(gca,"TickLabelInterpreter","latex")
25 xticklabels({"", "\frac{max\_norm}{10000}$", "\frac{max\_norm}{1000}$", "\frac{max\_
        ↪ norm}{100}$", "\frac{max\_norm}{10}$"})
26 set(gcf, 'Units', 'Normalized', 'OuterPosition', [0, 0.04, 0.8, 0.4]);
27 if save_image
28     exportgraphics(gcf,strcat("q36_ssimsnr.png"), 'Resolution', 300)
29 end

```

Question 3.7

We can try Kaczmazr method under the presence of noise as well. We can use different regularization parameters to see how it affects the reconstruction quality. We will reconstruct the image using 5 different lambda values. These lambda values are $\sigma_1^2 \times 10^{-6}$, $\sigma_1^2 \times 10^{-5}$, $\sigma_1^2 \times 10^{-4}$, $\sigma_1^2 \times 10^{-3}$, and $\sigma_1^2 \times 10^{-2}$. The resulting reconstructions for each lambda value are shown in Figures 3.7.1 through 3.7.5.

Regularized Kaczmarz Recon, $\lambda = \sigma_1^2 10^{-6}$



Reference Image



Error Map

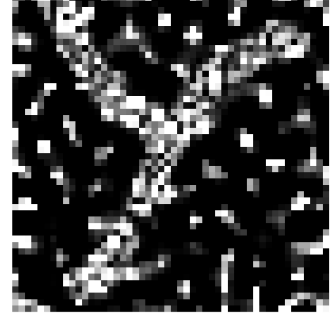


Figure 3.7.1: Reconstruction with Regularization Parameter $\lambda = \sigma_1^2 \times 10^{-6}$ with Noise

Regularized Kaczmarz Recon, $\lambda = \sigma_1^2 10^{-5}$



Reference Image



Error Map

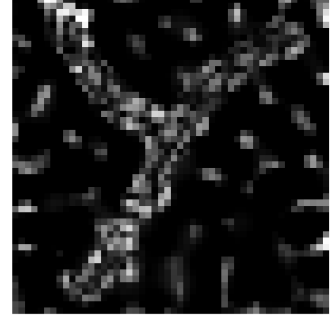
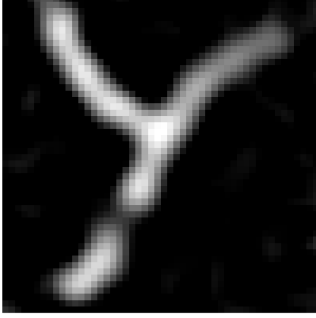


Figure 3.7.2: Reconstruction with Regularization Parameter $\lambda = \sigma_1^2 \times 10^{-5}$ with Noise

Regularized Kaczmarz Recon, $\lambda = \sigma_1^2 10^{-4}$



Reference Image

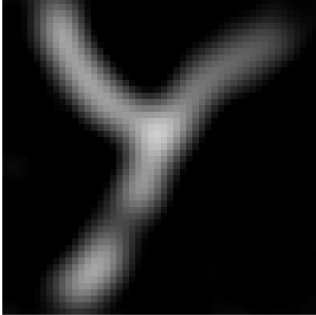


Error Map

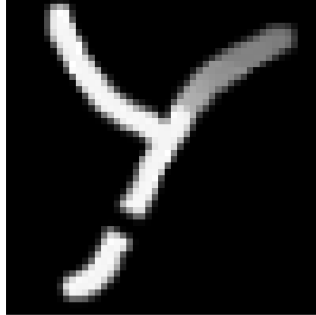


Figure 3.7.3: Reconstruction with Regularization Parameter $\lambda = \sigma_1^2 \times 10^{-4}$ with Noise

Regularized Kaczmarz Recon, $\lambda = \sigma_1^2 10^{-3}$



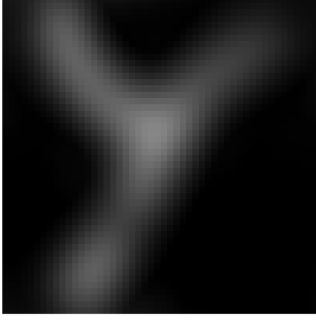
Reference Image



Error Map



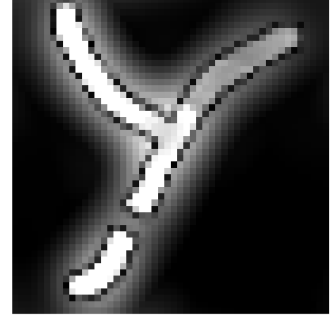
Figure 3.7.4: Reconstruction with Regularization Parameter $\lambda = \sigma_1^2 \times 10^{-3}$ with Noise

Regularized Kaczmarz Recon, $\lambda = \sigma_1^2 10^{-2}$ 

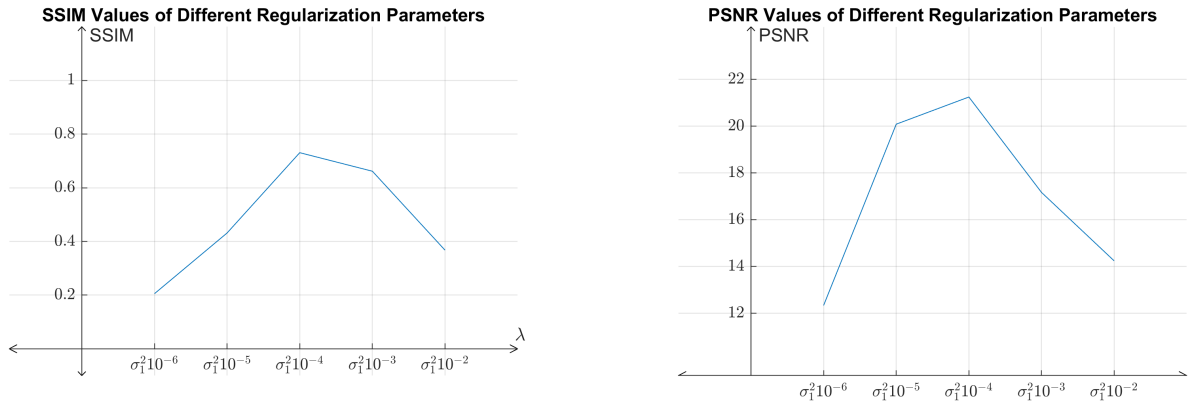
Reference Image



Error Map

Figure 3.7.5: Reconstruction with Regularization Parameter $\lambda = \sigma_1^2 \times 10^{-2}$ with Noise

We can again see that higher regularization parameters result in blurrier images where as low regularization parameters amplify the noise and result in worse image quality. The PSNR and SSIM values for each reconstruction is shown in Figure 3.7.6

Figure 3.7.6: SSIM and PSNR Values for Different Regularization Parameters λ with Noise

Lambda Value	$\sigma_1^2 \times 10^{-6}$	$\sigma_1^2 \times 10^{-5}$	$\sigma_1^2 \times 10^{-4}$	$\sigma_1^2 \times 10^{-3}$	$\sigma_1^2 \times 10^{-2}$
SSIM	0.2054	0.4317	0.7309	0.6618	0.3676
PSNR	12.3398	20.0858	21.2464	17.1686	14.2442

Table 3.7.1: SSIM and PSNR Values for Different Regularization Parameters λ with Noise

```

1  s1 = Sigma(1,1)^2;
2  lambda_rels = [10^-6 10^-5 10^-4 10^-3 10^-2];
3  PSNRs3_7 = zeros(1,numel(lambda_rels));
4  SSIMs3_7 = zeros(1,numel(lambda_rels));
5  for iter = 1:numel(lambda_rels)
6      S_reg = [S eye(size_S(1)).*sqrt(lambda_rels(iter)*s1)];
7      u_noisy_reg = [u_noisy ; zeros(size_S(1),1)*sqrt(lambda_rels(iter))];
8      c = kaczmarz_method(S_reg,u_noisy_reg,50,lambda_rels(iter)*s1,{0,0,0,0,0});
9      ima = reshape(c,50,50);
10     ima(ima<0) = 0;
11     plot_comparative_images(ima,m_ref,save_image,sprintf("Q37_comp_%i",iter),sprintf
        ↪ ("Regularized Kaczmarz Recon, %s = %s", "$\lambda$",strcat("$\sigma_1 ^2
        ↪ 10^{",num2str(iter-7)," } $")), "Reference Image",max_val)
12     PSNRs3_7(iter) = psnr(ima/max_val,m_ref/max_val);
13     SSIMs3_7(iter) = ssim(ima/max_val,m_ref/max_val);
14 end
15 figure();
16 subplot(1,2,1)
17 fineplot(1:numel(lambda_rels),SSIMs3_7,"SSIM Values of Different Regularization
        ↪ Parameters", "\lambda", "SSIM", [-1 6], [-0.1 1.2], "off", [800 400], "", "-");
18 set(gca,"TickLabelInterpreter","latex")
19 xticklabels({"", "$\sigma_1 ^2 10^{-6}$", "$\sigma_1 ^2 10^{-5}$", "$\sigma_1 ^2
        ↪ 10^{-4}$", "$\sigma_1 ^2 10^{-3}$", "$\sigma_1 ^2 10^{-2}$"})
20 subplot(1,2,2)
21 fineplot(1:numel(lambda_rels),PSNRs3_7,"PSNR Values of Different Regularization
        ↪ Parameters", "\lambda", "PSNR", [-1 6], [min(PSNRs3_7)-3 max(PSNRs3_7)+3], "off", [800
        ↪ 400], "", "-");
22 set(gca,"TickLabelInterpreter","latex")
23 xticklabels({"", "$\sigma_1 ^2 10^{-6}$", "$\sigma_1 ^2 10^{-5}$", "$\sigma_1 ^2
        ↪ 10^{-4}$", "$\sigma_1 ^2 10^{-3}$", "$\sigma_1 ^2 10^{-2}$"})
24 set(gcf, 'Units', 'Normalized', 'OuterPosition', [0, 0.04, 0.8, 0.4]);
25 if save_image
26     exportgraphics(gcf,strcat("q37_ssimsnr.png"), 'Resolution', 300)
27 end

```

Question 4

So far we worked with simulated data. In this question we will instead work with real MPI data obtained from OpenMPI's database [1]. The given data is of a phantom that is in the shape of a cone [4]. The obtained measurement is also noisy since we are working with real data. We will apply regularized Kaczmarz method to reconstruct the phantom. We will again use different λ 's to see how it affects the reconstruction quality.

Question 4.1

We load the data, this time it is already processed as opposed to the first question. We can plot the system matrix rows for three arbitrary locations, these rows being 5, 131, 1051.

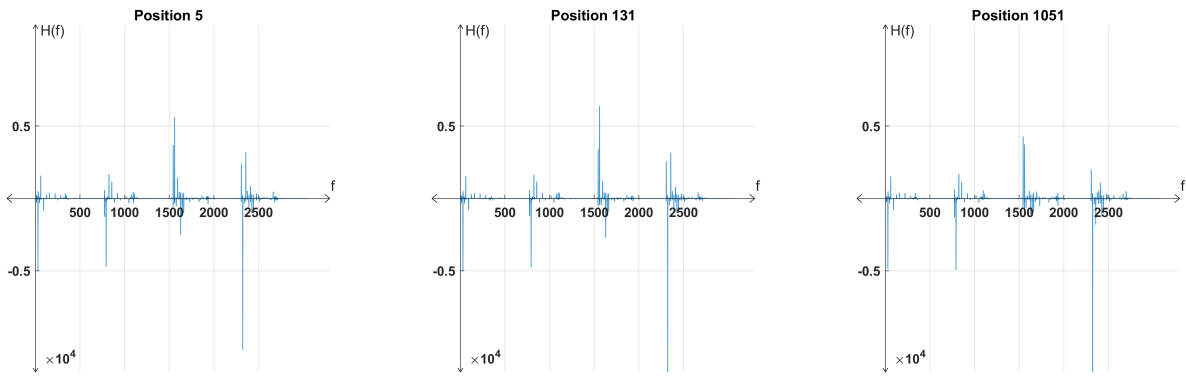


Figure 4.1.1: System Matrix Rows for Three Arbitrary Locations

```

1 data = load("su_openmpi.mat");
2 S_openmpi = data.S;
3 u_openmpi = data.u;
4 size_S_openmpi = size(S_openmpi);
5 figure;
6 subplot(1,3,1)
7 fineplot(1:size_S_openmpi(1),S_openmpi(:,5),"Position 5","f","H(f)",[-320
  ↳ 3300],[-12000 12000],"off",[1500 400],"","-");
8 subplot(1,3,2)
9 fineplot(1:size_S_openmpi(1),S_openmpi(:,131),"Position 131","f","H(f)",[-320
  ↳ 3300],[-12000 12000],"off",[1500 400],"","-");
10 subplot(1,3,3)
11 fineplot(1:size_S_openmpi(1),S_openmpi(:,1051),"Position 1051","f","H(f)",[-320
  ↳ 3300],[-12000 12000],"off",[1500 400],"","-");
12 set(gcf, 'Units', 'Normalized', 'OuterPosition', [0, 0, 1, 0.5]);
13 if save_image
14     exportgraphics(gcf,strcat("q41_sysmatrows.png"),'Resolution',300)
15 end

```

Question 4.2

We will need the singular value decomposition of the system matrix. We obtained the condition number of the matrix as: 2.1392×10^{10} . We can also see the singular values as shown in Figure 4.2.1

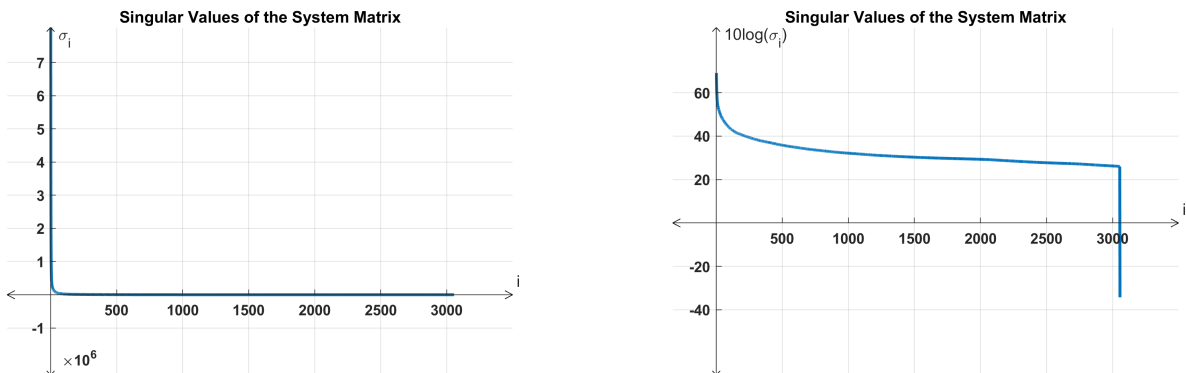


Figure 4.2.1: Singular Values of the System Matrix

```

1 [U_openmpi,Sigma_openmpi,V_openmpi] = svd(S_openmpi,'econ');
2 cond_num_openmpi = Sigma_openmpi(1,1)/Sigma_openmpi(end,end)
3 s_openmpi = diag(Sigma_openmpi);
4 figure();
5 subplot(1,2,1)
6 p = fineplot(1:numel(s_openmpi),s_openmpi,"Singular Values of the System
   ↳ Matrix","i","\sigma_i",[-330 3500],[-Sigma_openmpi(1,1)*0.3
   ↳ Sigma_openmpi(1,1)],"off",[1200 400],"","-");
7 p.LineWidth = 2;
8 subplot(1,2,2)
9 p = fineplot(1:numel(s_openmpi),10*log10(s_openmpi),"Singular Values of the System
   ↳ Matrix","i","10log(\sigma_i)",[-330 3500],[-70 90],"off",[1200 400],"","-");
10 p.LineWidth = 2;
11 set(gcf, 'Units', 'Normalized', 'OuterPosition', [0, 0, 0.9, 0.45]);
12 if save_image
13     exportgraphics(gcf,strcat("q42_sigma.png"),'Resolution',300)
14 end

```

Question 4.3

We can select $\lambda = \sigma_1 \sigma_N$ without thresholding. We obtain the 3D reconstruction of the phantom. To display the phantom, we will plot each cross-section of the phantom using MATLAB's montage function. The resulting reconstruction is shown in Figure 4.3.1.

OpenMPI Phantom, Regularized Kaczmarz Recon, $\lambda = \sigma_1 \sigma_N = 3.031 \times 10^3$



Figure 4.3.1: Reconstruction of the Phantom with $\lambda = \sigma_1 \sigma_N$

```

1 S_reg_openmpi = [S_openmpi eye(size_S_openmpi(1))];
2 u_reg_openmpi = [u_openmpi ; zeros(size_S_openmpi(1),1)];
3 lambda = s_openmpi(1)*s_openmpi(end);
4 c = modified_kaczmarz(S_reg_openmpi,u_reg_openmpi,lambda);
5 ima = reshape(c,37,37,37);
6 figure();
7 montage(reshape(ima,[37, 37, 1, 37]),'displayRange',[]);
8 title(strcat("OpenMPI Phantom, Regularized Kaczmarz Recon, $\lambda = \sigma_1$
  ↳ $\sigma_N$ = ",num2sci(lambda,"latex")), "Interpreter","latex")
9 set(gcf, 'Units', 'Normalized', 'OuterPosition', [0, 0, 1, 1]);
10 if save_image
11     exportgraphics(gcf,strcat("q43_phantom.png"),'Resolution',300)
12 end

```

Question 4.4

We can also select different λ values as: $\sigma_1^2 \times 10^{-6}$, $\sigma_1^2 \times 10^{-5}$, $\sigma_1^2 \times 10^{-4}$, $\sigma_1^2 \times 10^{-3}$, and $\sigma_1^2 \times 10^{-2}$. The resulting reconstructions for each lambda value are shown in Figures 4.4.1 through 4.4.5.

OpenMPI Phantom, Regularized Kaczmarz Recon, $\lambda = \sigma_1^2 10^{-6}$



Figure 4.4.1: Reconstruction of the Phantom with $\lambda = \sigma_1^2 \times 10^{-6}$

OpenMPI Phantom, Regularized Kaczmarz Recon, $\lambda = \sigma_1^2 10^{-5}$

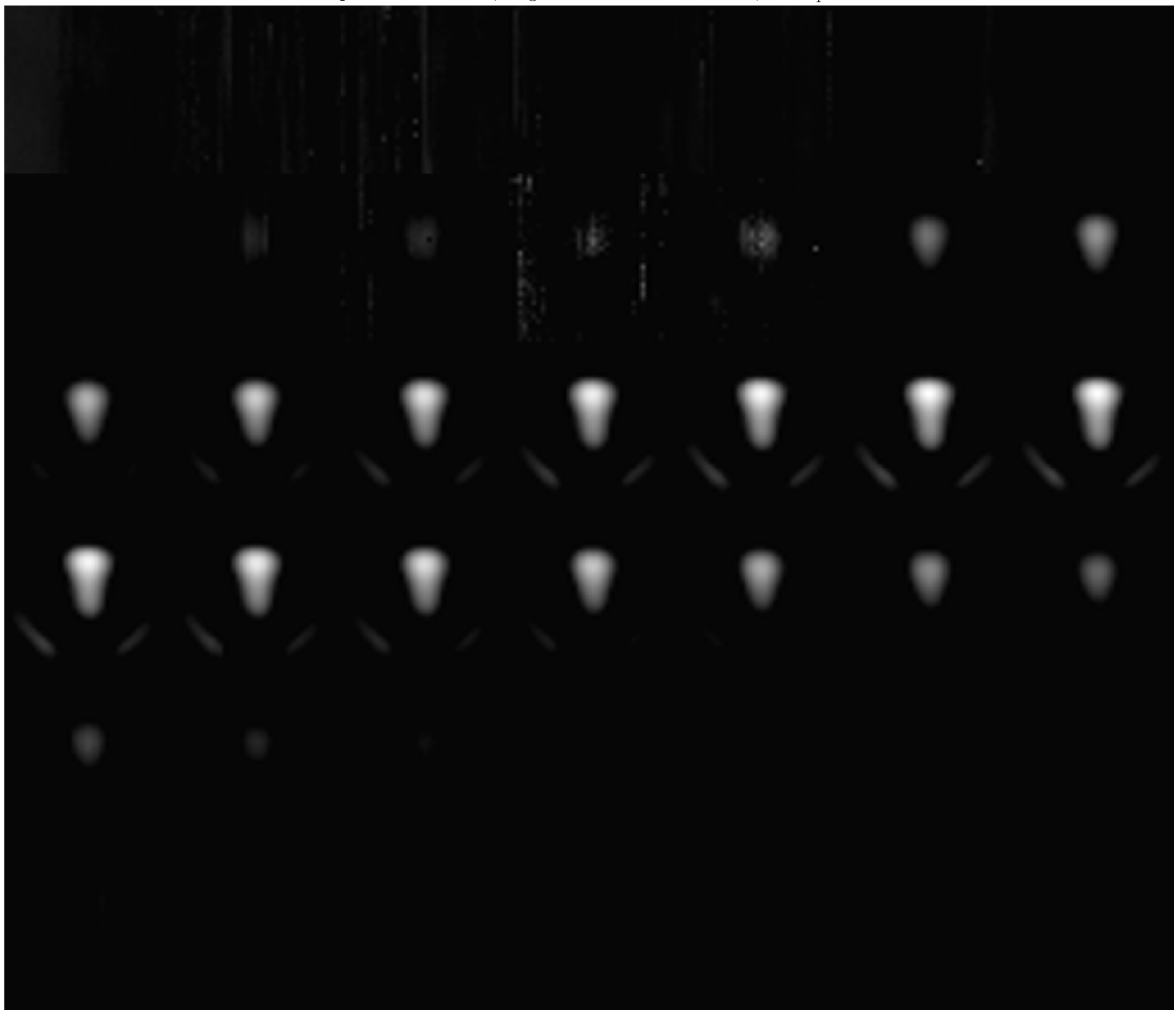


Figure 4.4.2: Reconstruction of the Phantom with $\lambda = \sigma_1^2 \times 10^{-5}$

OpenMPI Phantom, Regularized Kaczmarz Recon, $\lambda = \sigma_1^2 10^{-4}$

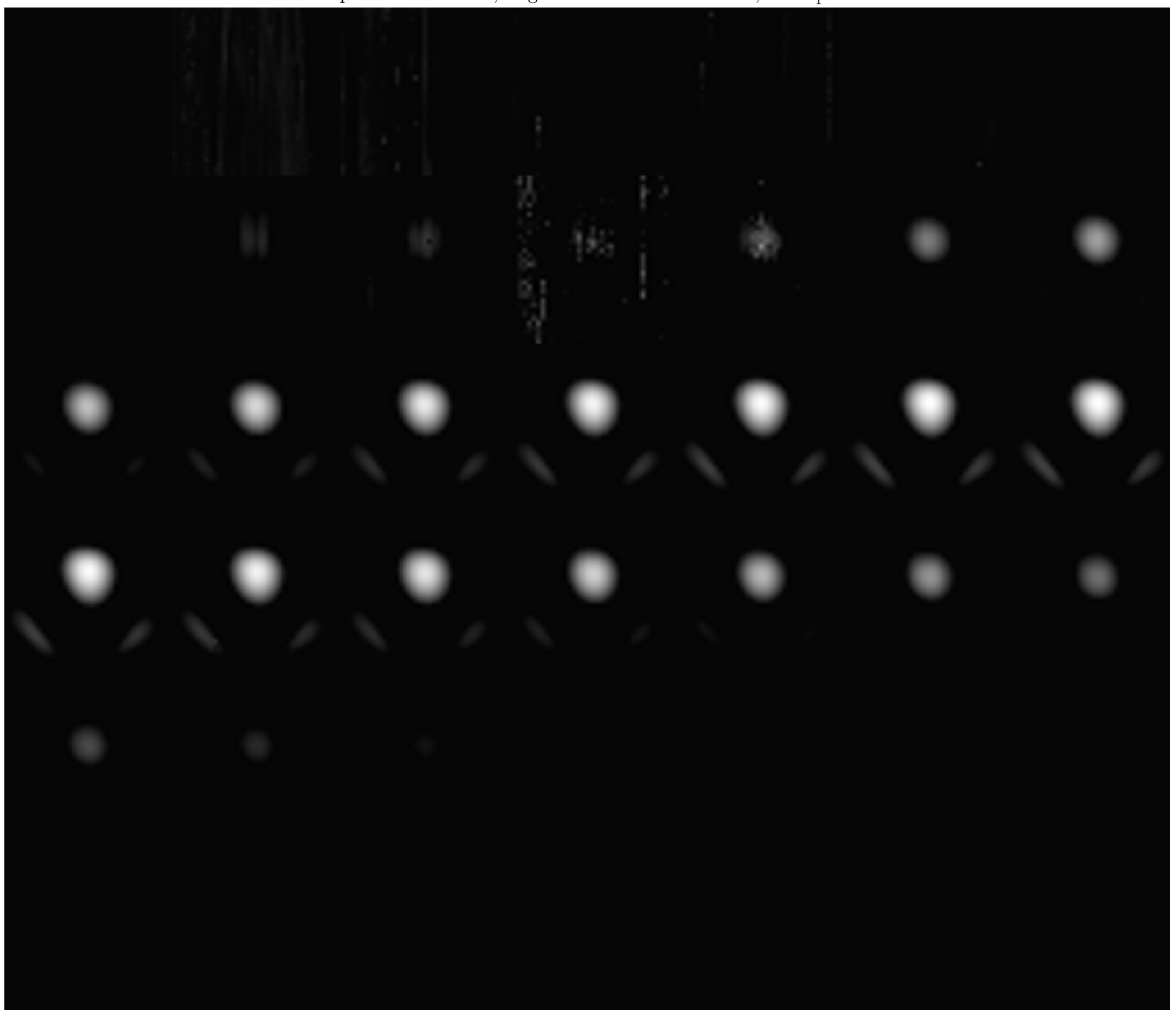


Figure 4.4.3: Reconstruction of the Phantom with $\lambda = \sigma_1^2 \times 10^{-4}$

OpenMPI Phantom, Regularized Kaczmarz Recon, $\lambda = \sigma_1^2 10^{-3}$



Figure 4.4.4: Reconstruction of the Phantom with $\lambda = \sigma_1^2 \times 10^{-3}$

OpenMPI Phantom, Regularized Kaczmarz Recon, $\lambda = \sigma_1^2 10^{-2}$



Figure 4.4.5: Reconstruction of the Phantom with $\lambda = \sigma_1^2 \times 10^{-2}$

The reconstructions are very blurry due to the high noise level in the measurements. Higher regularization parameters result in blurrier images, whereas lower regularization parameters amplify the noise and result in worse image quality. The main trade-off here is between noise amplification and information. $\lambda = \sigma_1^2 \times 10^{-6}$ seems to give the best reconstruction quality.

```

1  lambda_rels2 = [10^-6 10^-5 10^-4 10^-3 10^-2];
2  for i = 1:numel(lambda_rels2)
3      c = modified_kaczmarz(S_reg_openmpi,u_reg_openmpi,lambda_rels2(i) *
        ↳ (Sigma_openmpi(1,1)^2));
4      ima = reshape(c,37,37,37);
5      ima(ima<0) = 0;
6      figure();
7      montage(reshape(ima,[37, 37, 1, 37]),'displayRange',[]);
8      title(strcat("OpenMPI Phantom, Regularized Kaczmarz Recon, $\lambda = \sigma_1^2$
        ↳ 10^{-",num2str(i-7),"} $"), "Interpreter", "latex")
9      set(gcf, 'Units', 'Normalized', 'OuterPosition', [0, 0, 1, 1]);
10     if save_image
11         exportgraphics(gcf,strcat("q44",num2str(i), "_phantom.png"), 'Resolution',300)
12     end
13 end

```

Conclusion

In this report, we have implemented and analyzed the Kaczmarz method for image reconstruction in MPI. We have seen how thresholding and regularization can be used to improve the reconstruction quality. We have also seen how noise affects the reconstruction and how regularization can help mitigate the effects of noise. Finally, we have applied the regularized Kaczmarz method to real MPI data and analyzed the results. The main takeaway from this report is that there is a trade-off between noise amplification and information in image reconstruction, and finding the right balance is crucial for obtaining good reconstruction quality.

References

- [1] Open MPI Data, “Datasets.” <https://magneticparticleimaging.github.io/OpenMPIData.jl/latest/datasets.html>. Accessed: 2026-04-12.
- [2] Dropbox, “SM_25nm_50x50.mat.” https://www.dropbox.com/scl/fi/n9xeny0dqq7i522mxk9hn/SM_25nm_50x50.mat?rlkey=i1pcnciqap1uoc15ttl713s8o. MAT file, accessed: 2026-04-12.
- [3] Dropbox, “meas_25nm_phantom.mat.” https://www.dropbox.com/scl/fi/n9xeny0dqq7i522mxk9hn/meas_25nm_phantom.mat?rlkey=i1pcnciqap1uoc15ttl713s8o. MAT file, accessed: 2026-04-12.
- [4] Open MPI Data, “Phantoms.” <https://magneticparticleimaging.github.io/OpenMPIData.jl/latest/phantoms.html>. Accessed: 2026-04-12.

```
save_image = 1;
```

Question 1

Question 1.1

```
data = load("SM_25nm_50x50.mat");
SM = data.SM;
figure;
subplot(2,3,1)
fineplot(1:1632,abs(SM(1,:,5)),"Coil 1, Position 5","f","H(f)",[-170 1700],[0
16*(10^8)],"off",[2000 400],"","-");
subplot(2,3,2)
fineplot(1:1632,abs(SM(1,:,131)),"Coil 1, Position 131","f","H(f)",[-170 1700],
[0 16*(10^8)],"off",[2000 400],"","-");
subplot(2,3,3)
fineplot(1:1632,abs(SM(1,:,1051)),"Coil 1, Position 1051","f","H(f)",[-170 1700],
[0 16*(10^8)],"off",[2000 400],"","-");
subplot(2,3,4)
fineplot(1:1632,abs(SM(2,:,5)),"Coil 2, Position 5","f","H(f)",[-170 1700],[0
16*(10^8)],"off",[2000 400],"","-");
subplot(2,3,5)
plot(abs(SM(2,:,131)))
fineplot(1:1632,abs(SM(2,:,131)),"Coil 2, Position 131","f","H(f)",[-170 1700],
[0 16*(10^8)],"off",[2000 400],"","-");
subplot(2,3,6)
fineplot(1:1632,abs(SM(2,:,1051)),"Coil 2, Position 1051","f","H(f)",[-170 1700],
[0 16*(10^8)],"off",[2000 400],"","-");
set(gcf, 'Units', 'Normalized', 'OuterPosition', [0, 0, 1, 0.6]);
if save_image
    exportgraphics(gcf,strcat("q11_sysmatrows.png"),'Resolution',300)
end
```

Question 1.2

```
S = SM(:,1:end/2,:);
S = reshape(S,2*816,2500);
S = [real(S); imag(S)];
figure;
subplot(1,3,1)
fineplot(1:3264,S(:,5),"Position 5","f","H(f)",[-320 3300],[-1.5
1.5]*(10^9),"off",[1500 400],"","-");
subplot(1,3,2)
fineplot(1:3264,S(:,131),"Position 131","f","H(f)",[-320 3300],[-1.5
1.5]*(10^9),"off",[1500 400],"","-");
subplot(1,3,3)
fineplot(1:3264,S(:,1051),"Position 1051","f","H(f)",[-320 3300],[-1.5
1.5]*(10^9),"off",[1500 400],"","-");
set(gcf, 'Units', 'Normalized', 'OuterPosition', [0, 0, 1, 0.5]);
if save_image
    exportgraphics(gcf,strcat("q12_sysmatrows.png"),'Resolution',300)
end
save("S.mat","S")
```

Question 1.3

```
data = load("meas_25nm_phantom.mat");
m_ref = data.phantom_ref;
meas = data.meas;
figure;
imshow(m_ref,[])
title("Reference Image")
max_val = max(m_ref(:));
set(gcf, 'Units', 'Normalized', 'OuterPosition', [0, 0, 0.5, 0.5]);
if save_image
    exportgraphics(gcf, strcat("q13_ref.png"), 'Resolution', 300)
end
u = meas(:,1:end/2);
u = reshape(u, 2*816, 1);
u = [real(u); imag(u)];
figure;
fineplot(1:length(u), u, "Measurement", "f", "c(f)", [-330 3300], [-4
16]*(10^10), "off", [800 600], "", "-");
set(gcf, 'Units', 'Normalized', 'OuterPosition', [0, 0, 0.6, 0.45]);
if save_image
    exportgraphics(gcf, strcat("q13_meas.png"), 'Resolution', 300)
end
```

Question 2

Question 2.1

```
S = load("S.mat");
S = S.S;
[U, Sigma, V] = svd(S, 'econ');
cond_num = Sigma(1,1)/Sigma(end,end)
cond_num2 = cond(S);
fprintf("Condition Number from Singular Values : %s, Condition Number from
MATLAB Function: %s", num2sci(cond_num), num2sci(cond_num2))
fprintf("Largest Singular Value: %s, Smallest Singular Value:
%s", num2sci(Sigma(1,1)), num2sci(Sigma(end,end)))
figure;
subplot(1,2,1)
p = fineplot(1:2500, sum(Sigma), "Singular Values", "i", "\sigma_i", [-250 2600], [-4
19]*(10^10), "off", [800 400], "", "-");
p.LineWidth = 3;
subplot(1,2,2)
p = fineplot(1:2500, 10*log10(sum(Sigma)), "Singular Values", "i", "10log(\sigma_i)",
[-250 2600], [-80 120], "off", [1200 400], "", "-");
p.LineWidth = 3;
set(gcf, 'Units', 'Normalized', 'OuterPosition', [0, 0, 0.9, 0.45]);
if save_image
    exportgraphics(gcf, strcat("q21_sigma.png"), 'Resolution', 300)
end
```

Question 2.2

```

s = diag(Sigma);
sinv = zeros(size(s));
tol = 1e-12; % or use a tolerance based on max(s)
sinv(s > tol) = 1 ./ s(s > tol);
Sigma_inv = diag(sinv);
c = V * Sigma_inv * ctranspose(U) * u;
ima = reshape(c,50,50);
ima(ima<0) = 0;
plot_comparetative_images(ima,m_ref,save_image,"q22_comp.png","SVD
Recon","Reference Image",max_val)
PSNR2_2 = psnr(ima/max_val,m_ref/max_val);
SSIM2_2 = ssim(ima/max_val,m_ref/max_val);

```

Question 2.3

```

cond_num_desired = 100;
[val,M] = min(abs(s-(s(1)/cond_num_desired)))
Sigma_new = Sigma(1:M,1:M);
U_new = U(:,1:M);
V_new = V(:,1:M);
Sigma_new_inv = invert_sigma(Sigma_new);
c = V_new * Sigma_new_inv * ctranspose(U_new) * u;
ima = reshape(c,50,50);
ima(ima<0) = 0;
plot_comparetative_images(ima,m_ref,save_image,"q23_comp.png",strcat("Truncated
SVD Recon,  $\frac{\sigma_1}{\sigma_M}$  = ",num2str(cond_num_desired)),"Reference
Image",max_val)
PSNR2_3 = psnr(ima/max_val,m_ref/max_val);
SSIM2_3 = ssim(ima/max_val,m_ref/max_val);

```

Question 2.4

```

cond_nums_desired = [1 5 10 100 10^3 10^4 10^5 10^6 10^7];
PSNRs2_4 = zeros(size(cond_nums_desired));
SSIMs2_4 = PSNRs2_4;
for iteration = 1:length(cond_nums_desired)

    [val,M] = min(abs(s-(s(1)/cond_nums_desired(iteration))));
    Sigma_new = Sigma(1:M,1:M);
    U_new = U(:,1:M);
    V_new = V(:,1:M);

    Sigma_new_inv = invert_sigma(Sigma_new);
    c = V_new * Sigma_new_inv * ctranspose(U_new) * u;
    ima = reshape(c,50,50);
    ima(ima<0) = 0;

    plot_comparetative_images(ima,m_ref,save_image,strcat("q24",num2str(iteration)),"_co
mp.png"),strcat("Truncated SVD Recon,  $\frac{\sigma_1}{\sigma_M}$  = ",num2str(cond_nums_desired(iteration))),"Reference Image",max_val)
    SSIMs2_4(iteration) = ssim(ima/max_val,m_ref/max_val);
    PSNRs2_4(iteration) = psnr(ima/max_val,m_ref/max_val);
end

```

```

figure();
subplot(1,2,1)
finesemilogx(cond_nums_desired,SSIMs2_4,"off","SSIMs for Different
Truncations","\sigma_1/\sigma_m","SSIM",[0 9.4],[0 1],"off",[800 400],"","-")
subplot(1,2,2)
finesemilogx(cond_nums_desired,PSNRs2_4,"off","PSNRs for Different
Truncations","\sigma_1/\sigma_m","PSNR",[0 9.4],[min(PSNRs2_4)-1
max(PSNRs2_4)+1],"off",[800 400],"","-")
set(gcf, 'Units', 'Normalized', 'OuterPosition', [0, 0, 0.8, 0.4]);
if save_image
    exportgraphics(gcf, strcat("q24_ssimsnr.png"), 'Resolution', 300)
end

```

Question 2.5

Analytical proof required for this question. No scripts required.

Question 2.6

```

lambda = Sigma(1,1) * Sigma(end,end)
s_reg = (s.^2 + lambda)./s;
Sigma_reg = diag(s_reg);
Sigma_reg_inv = invert_sigma(Sigma_reg);
c = V * Sigma_reg_inv * ctranspose(U) * u;
ima = reshape(c,50,50);
ima(ima<0) = 0;

figure;
subplot(1,2,1)
p1 = fineplot(1:numel(s_reg),s_reg,"Filtered and Original Singular
Values","n","\sigma_i",[-250 2600],[-2 19]*(10^10),"on",[1700 400],"Filtered
Singular Values","-");
p2 = fineplot(1:numel(s),s,"Filtered and Original Singular
Values","n","\sigma_i",[-250 2600],[-2 19]*(10^10),"on",[1700 400],"Original
Singular Values","--");
p1.LineWidth = 3;
p2.LineWidth = 3;
legend("Location","north")
subplot(1,2,2)
p1 = fineplot(1:numel(s_reg),10*log10(s_reg),"Filtered and Original Singular
Values","n","\sigma_i",[-250 2600],[-70 120],"on",[1700 400],"Filtered Singular
Values","-");
p2 = fineplot(1:numel(s),10*log10(s),"Filtered and Original Singular
Values","n","10log(\sigma_i)",[-250 2600],[-70 120],"on",[1700 400],"Original
Singular Values","--");
p1.LineWidth = 3;
p2.LineWidth = 3;
legend("Location","south")
set(gcf, 'Units', 'Normalized', 'OuterPosition', [0, 0, 0.9, 0.45]);
if save_image
    exportgraphics(gcf, strcat("q26_sigma.png"), 'Resolution', 300)
end

```

```

plot_comparative_images(ima,m_ref,save_image,"q26_comp.png",strcat("Filtered SVD
Recon, $\lambda$ = ",num2sci(lambda,"latex")), "Reference Image",max_val)
PSNR2_6 = psnr(ima/max_val,m_ref/max_val);
SSIM2_6 = ssim(ima/max_val,m_ref/max_val);

```

Question 2.7

```

lambda_sweep = logspace(-4,17,22);
PSNRs_Sweep = zeros(size(lambda_sweep));
SSIMs_Sweep = PSNRs_Sweep;
for iteration = 1:numel(lambda_sweep)
    s_reg = (s.^2 + lambda_sweep(iteration))./s;
    Sigma_reg = diag(s_reg);
    Sigma_reg_inv = invert_sigma(Sigma_reg);
    c = V * Sigma_reg_inv * ctranspose(U) * u;
    ima = reshape(c,50,50);
    ima(ima<0) = 0;
    PSNRs_Sweep(iteration) = psnr(ima/max_val,m_ref/max_val);
    SSIMs_Sweep(iteration) = ssim(ima/max_val,m_ref/max_val);
end
figure;
subplot(1,2,1)
finesemilogx(lambda_sweep,SSIMs_Sweep,"off","SSIM for Different Lambda
Values", "\lambda", "SSIM", [-4 17],[0 1],"off",[800 400],"", "-");
subplot(1,2,2)
finesemilogx(lambda_sweep,PSNRs_Sweep,"off","PSNR for Different Lambda
Values", "\lambda", "PSNR", [-4 17],[-50 50],"off",[800 400],"", "-");
set(gcf,'position',[0, 0, 800 , 400]);
set(gcf, 'Units', 'Normalized', 'OuterPosition', [0, 0.04, 0.8, 0.4]);
if save_image
    exportgraphics(gcf,strcat("q27_ssimsnr.png"), 'Resolution',300)
end
lambda = 4.171 * (10^9);
PSNRs2_7 = zeros(1,3);
SSIMs2_7 = zeros(1,3);
lambdas2 = [lambda/10 lambda lambda*10];
for iteration = 1:3
    s_reg = (s.^2 + lambdas2(iteration))./s;
    Sigma_reg = diag(s_reg);
    Sigma_reg_inv = invert_sigma(Sigma_reg);
    c = V * Sigma_reg_inv * ctranspose(U) * u;
    ima = reshape(c,50,50);
    ima(ima<0) = 0;

    PSNRs2_7(iteration) = psnr(ima/max_val,m_ref/max_val);
    SSIMs2_7(iteration) = ssim(ima/max_val,m_ref/max_val);

plot_comparative_images(ima,m_ref,save_image,strcat("q27_",num2str(iteration), "_c
omp.png"),strcat("Filtered SVD Recon, $\lambda$ =
",num2sci(lambdas2(iteration),"latex")), "Reference Image",max_val)
end

```

Question 2.8

```

residual_norms = [];
solution_norms = [];
lambda_sweep2 = (s(1)*s(end))*logspace(0,7,8);
for lambda = lambda_sweep2
    s_reg = (s.^2 + lambda)./s;
    Sigma_reg = diag(s_reg);
    Sigma_reg_inv = invert_sigma(Sigma_reg);
    c = V * Sigma_reg_inv * ctranspose(U) * u;
    ima = reshape(c,50,50);
    ima(ima<0) = 0;
    residual_norm = norm(S*c - u);
    solution_norm = norm(c);
    residual_norms = [residual_norms residual_norm];
    solution_norms = [solution_norms solution_norm];
end
lambda = lambda_sweep2(3)
figure;
fineloglog(residual_norms, solution_norms, "off","L-Curve","Residual
Norm","Solution Norm",[0 6],[0.4 0.6],"on",[400 400],"","-o");
plot(residual_norms(3),solution_norms(3),"p","LineWidth",2)
grid on
set(gcf, 'Units', 'Normalized', 'OuterPosition', [0, 0.04, 0.7, 0.7]);
if save_image
    exportgraphics(gcf,strcat("q28_lcurve.png"),'Resolution',300)
end
s_reg = (s.^2 + lambda)./s;
Sigma_reg = diag(s_reg);
Sigma_reg_inv = invert_sigma(Sigma_reg);
c = V * Sigma_reg_inv * ctranspose(U) * u;

ima = reshape(c,50,50);
ima(ima<0) = 0;
PSNR2_8 = psnr(ima/max_val,m_ref/max_val);
SSIM2_8 = ssim(ima/max_val,m_ref/max_val);
plot_comparative_images(ima,m_ref,save_image,strcat("q28_comp.png"),strcat("Filt
red SVD Recon, $\lambda$ = ",num2sci(lambda,"latex")), "Reference Image",max_val)

```

```

Questions = {'2.2','2.3','2.6','2.8','2.7 /10','2.7','2.7
x10','2.4.1','2.4.2','2.4.3','2.4.4','2.4.5','2.4.6','2.4.7','2.4.8','2.4.9'};
SSIMs = num2cell([SSIM2_2 SSIM2_3 SSIM2_6 SSIM2_8 SSIMs2_7 SSIMs2_4]);
PSNRs = num2cell([PSNR2_2 PSNR2_3 PSNR2_6 PSNR2_8 PSNRs2_7 PSNRs2_4]);
columns = {'Questions';'SSIMs';'PSNRs'};
tableq1 = [columns,[Questions;SSIMs;PSNRs]];
disp(tableq1(:,1:9))
disp(tableq1(:,9:end))

```

Question 3

Question 3.1

```

S = load("S.mat");
S = S.S;
size_S = size(S);

```



```

norms = sqrt(sum(S.^2,2));
clf;
figure(1: numel(norms), norms, "Norms of Each Row of the System Matrix S", "i", "||S_i||_2", [-0.1*size_S(1) size_S(1)], [-2*10^10 14*10^10], "off", [400 400], "", "-");
set(gcf, 'Units', 'Normalized', 'OuterPosition', [0, 0.04, 0.5, 0.5]);
if save_image
    exportgraphics(gcf, strcat("q31_norms.png"), 'Resolution', 300)
end
max_norm = max(norms);
thresh1 = max_norm/10;
S_thresh = S(norms>thresh1,:);
u_thresh = u(norms>thresh1);

```

Question 3.2

```

rng(67); %prevents differences between executions
c = kaczmarz_method(S_thresh, u_thresh, 50, 0, {1, save_image, max_val, m_ref, "$\frac{\max\_norm}{10}$"}, [0 max_val]);

```

Question 3.3

```

threshs = [(10^-4) (10^-3) (10^-2) (10^-1)];
PSNRs3_3 = zeros(size(threshs));
SSIMs3_3 = zeros(size(threshs));
for iter = 1: numel(threshs)
    S_thresh_loop = S(norms>threshs(iter)*max_norm,:);
    u_thresh_loop = u(norms>threshs(iter)*max_norm);
    rng(67)
    c = kaczmarz_method(S_thresh_loop, u_thresh_loop, 50, 0, {0, 0, 0, 0, 0, 0});
    ima = reshape(c, 50, 50);
    ima(ima<0) = 0;

    plot_comparative_images(ima, m_ref, save_image, sprintf("Q33_comp_%i", iter), sprintf(
        "Kaczmarz Recon, Threshold = %s", strcat("$\frac{\max\_norm}{", num2str(1/
        threshs(iter)), " } $")), "Reference Image", max_val)
    PSNRs3_3(iter) = psnr(ima/max_val, m_ref/max_val);
    SSIMs3_3(iter) = ssim(ima/max_val, m_ref/max_val);
end
figure();
subplot(1,2,1)
figure(1: numel(threshs), SSIMs3_3, "SSIM Values for Different Thresholds", "Threshold", "SSIM", [-1 5], [-0.1 1.2], "off", [800 400], "", "-");
set(gca, "TickLabelInterpreter", 'latex')
xticklabels({"", "$\frac{\max\_norm}{10000}$", "$\frac{\max\_norm}{1000}$", "$\frac{\max\_norm}{100}$", "$\frac{\max\_norm}{10}$"})
subplot(1,2,2)
figure(1: numel(threshs), PSNRs3_3, "PSNR Values for Different Thresholds", "Threshold", "PSNR", [-1 5], [min(PSNRs3_3)-3 max(PSNRs3_3)+3], "off", [800 400], "", "-");
set(gca, "TickLabelInterpreter", 'latex')
xticklabels({"", "$\frac{\max\_norm}{10000}$", "$\frac{\max\_norm}{1000}$", "$\frac{\max\_norm}{100}$", "$\frac{\max\_norm}{10}$"})
set(gcf, 'Units', 'Normalized', 'OuterPosition', [0, 0.04, 0.8, 0.4]);

```

```

if save_image
    exportgraphics(gcf,strcat("q33_ssimsnr.png"),'Resolution',300)
end

```

Question 3.4

```

rng(67); %prevents differences between executions
lambda = Sigma(1,1) * Sigma(end,end);
size_S = size(S);
S_reg = [S eye(size_S(1)).*sqrt(lambda)];
u_reg = [u ; zeros(size_S(1),1)];
kaczmarz_method(S_reg,u_reg,50,lambda,{1,save_image,max_val,m_ref,"$\frac{\max\_norm}{10}$",[0 max_val]});

```

```

c2 = kaczmarz_method(S_reg,u_reg,50,lambda,{0,0,0,0,0,[ ]});
ima2_1 = reshape(c2,50,50);
ima2_1(ima2_1<0) = 0;
c = regularized_kaczmarz(S,u,lambda);
ima2_2 = reshape(c,50,50);
ima2_2(ima2_2<0) = 0;
subplot(1,2,1)
imshow(ima2_2,[ ])
title("new")
subplot(1,2,2)
imshow(ima2_1,[ ])
title("old")
ssim(ima2_2/max_val,m_ref/max_val)

```

Question 3.5

```

s1 = Sigma(1,1)^2;
lambda_rels = [10^-6 10^-5 10^-4 10^-3 10^-2];
PSNRs3_5 = zeros(1,numel(lambda_rels));
SSIMs3_5 = zeros(1,numel(lambda_rels));
for iter = 1:numel(lambda_rels)
    S_reg = [S eye(size_S(1)).*sqrt(lambda_rels(iter))];
    c = kaczmarz_method(S_reg,u_reg,50,lambda_rels(iter)*s1,{0,0,0,0,0,0});
    ima = reshape(c,50,50);
    ima(ima<0) = 0;

    plot_comparative_images(ima,m_ref,save_image,sprintf("Q35_comp_%i",iter),sprintf(
        "Regularized Kaczmarz Recon, %s = %s","$\lambda$",strcat("$\sigma_1^2 10^{",num2str(iter-7)," } $")), "Reference Image",max_val)
    PSNRs3_5(iter) = psnr(ima/max_val,m_ref/max_val);
    SSIMs3_5(iter) = ssim(ima/max_val,m_ref/max_val);
end
figure();
subplot(1,2,1)
fineplot(1:numel(lambda_rels),SSIMs3_5,"SSIM Values of Different Regularization Parameters", "\lambda", "SSIM", [-1 6], [-0.1 1.2], "off", [800 400], "", "-");
set(gca,"TickLabelInterpreter","latex')
xticklabels({"", "$\sigma_1^2 10^{-6}$", "$\sigma_1^2 10^{-5}$", "$\sigma_1^2 10^{-4}$", "$\sigma_1^2 10^{-3}$", "$\sigma_1^2 10^{-2}$"})

```

```

subplot(1,2,2)
fineplot(1:numel(lambda_rels),PSNRs3_5,"PSNR Values of Different Regularization
Parameters","\lambda","PSNR",[-1 6],[min(PSNRs3_5)-3 max(PSNRs3_5)+3],"off",[800
400],"","-");
set(gca,"TickLabelInterpreter",'latex')
xticklabels({"","\sigma_1^2 10^{-6}$","\sigma_1^2 10^{-5}$","\sigma_1^2
10^{-4}$","\sigma_1^2 10^{-3}$","\sigma_1^2 10^{-2}$"})
set(gcf, 'Units', 'Normalized', 'OuterPosition', [0, 0.04, 0.8, 0.4]);
if save_image
    exportgraphics(gcf,strcat("q35_ssimsnr.png"),'Resolution',300)
end

```

Question 3.6

```

rng(67)
u_noisy = u + normrnd(0,max_norm/1000,size(u));
threshs = [(10^-4) (10^-3) (10^-2) (10^-1)];
PSNRs3_6 = zeros(size(threshs));
SSIMs3_6 = zeros(size(threshs));
for iter = 1:numel(threshs)
    S_thresh_loop = S(norms>threshs(iter)*max_norm,:);
    u_thresh_loop = u_noisy(norms>threshs(iter)*max_norm);
    rng(67)
    c = kaczmarz_method(S_thresh_loop,u_thresh_loop,50,0,{0,0,0,0,0,0});
    ima = reshape(c,50,50);
    ima(ima<0) = 0;

    plot_comparative_images(ima,m_ref,save_image,sprintf("Q36_comp_%i",iter),sprintf(
    "Kaczmarz Recon, Threshold = %s",strcat("\frac{max\_norm}{",num2str(1/
    threshs(iter)), " } $")), "Reference Image",max_val)
    PSNRs3_6(iter) = psnr(ima/max_val,m_ref/max_val);
    SSIMs3_6(iter) = ssim(ima/max_val,m_ref/max_val);
end
figure();
subplot(1,2,1)
fineplot(1:numel(threshs),SSIMs3_6,"SSIM Values for Different
Thresholds","Threshold","SSIM",[-1 5],[-0.1 1.2],"off",[800 400],"","-");
set(gca,"TickLabelInterpreter",'latex')
xticklabels({"","\frac{max\_norm}{10000}$","\frac{max\_norm}{1000}$","\frac{max\_norm}{100}$",
"\frac{max\_norm}{10}$"})
subplot(1,2,2)
fineplot(1:numel(threshs),PSNRs3_6,"PSNR Values for Different
Thresholds","Threshold","PSNR",[-1 5],[min(PSNRs3_6)-3 max(PSNRs3_6)+3],"off",
[800 400],"","-");
set(gca,"TickLabelInterpreter",'latex')
xticklabels({"","\frac{max\_norm}{10000}$","\frac{max\_norm}{1000}$","\frac{max\_norm}{100}$",
"\frac{max\_norm}{10}$"})
set(gcf, 'Units', 'Normalized', 'OuterPosition', [0, 0.04, 0.8, 0.4]);
if save_image
    exportgraphics(gcf,strcat("q36_ssimsnr.png"),'Resolution',300)
end

```

Question 3.7

```

s1 = Sigma(1,1)^2;
lambda_rels = [10^-6 10^-5 10^-4 10^-3 10^-2];
PSNRs3_7 = zeros(1,numel(lambda_rels));
SSIMs3_7 = zeros(1,numel(lambda_rels));
for iter = 1:numel(lambda_rels)
    S_reg = [S eye(size_S(1)).*sqrt(lambda_rels(iter)*s1)];
    u_noisy_reg = [u_noisy ; zeros(size_S(1),1)*sqrt(lambda_rels(iter))];
    c = kaczmarz_method(S_reg,u_noisy_reg,50,lambda_rels(iter)*s1,{0,0,0,0,0,0});
    ima = reshape(c,50,50);
    ima(ima<0) = 0;

    plot_comparetive_images(ima,m_ref,save_image,sprintf("Q37_comp_%i",iter),sprintf(
    "Regularized Kaczmarz Recon, %s = %s", "$\lambda$",strcat("$\sigma_1 ^2
    10^{" ,num2str(iter-7)," } $")), "Reference Image",max_val)
    PSNRs3_7(iter) = psnr(ima/max_val,m_ref/max_val);
    SSIMs3_7(iter) = ssim(ima/max_val,m_ref/max_val);
end
figure();
subplot(1,2,1)
fineplot(1:numel(lambda_rels),SSIMs3_7,"SSIM Values of Different Regularization
Parameters", "\lambda", "SSIM", [-1 6], [-0.1 1.2], "off", [800 400], "", "-");
set(gca,"TickLabelInterpreter","latex')
xticklabels({"", "$\sigma_1 ^2 10^{-6}$", "$\sigma_1 ^2 10^{-5}$", "$\sigma_1 ^2
10^{-4}$", "$\sigma_1 ^2 10^{-3}$", "$\sigma_1 ^2 10^{-2}$"})
subplot(1,2,2)
fineplot(1:numel(lambda_rels),PSNRs3_7,"PSNR Values of Different Regularization
Parameters", "\lambda", "PSNR", [-1 6], [min(PSNRs3_7)-3 max(PSNRs3_7)+3], "off", [800
400], "", "-");
set(gca,"TickLabelInterpreter","latex')
xticklabels({"", "$\sigma_1 ^2 10^{-6}$", "$\sigma_1 ^2 10^{-5}$", "$\sigma_1 ^2
10^{-4}$", "$\sigma_1 ^2 10^{-3}$", "$\sigma_1 ^2 10^{-2}$"})
set(gcf, 'Units', 'Normalized', 'OuterPosition', [0, 0.04, 0.8, 0.4]);
if save_image
    exportgraphics(gcf,strcat("q37_ssimsnr.png"), 'Resolution', 300)
end

```

```

questions = {"SSIMs 3.3"; "PSNRs 3.3"; "SSIMs 3.5"; "PSNRs 3.5"; "SSIMs 3.6"; "PSNRs
3.6"; "SSIMs 3.7"; "PSNRs 3.7"};
tableq2 = [questions num2cell([[SSIMs3_3 0];[PSNRs3_3 0];SSIMs3_5;PSNRs3_5;
[SSIMs3_6 0];[PSNRs3_6 0];SSIMs3_7;PSNRs3_7])];
disp(tableq2)

```

Question 4

Question 4.1

```

data = load("su_openmpi.mat");
S_openmpi = data.S;
u_openmpi = data.u;
size_S_openmpi = size(S_openmpi);
figure;
subplot(1,3,1)

```

```

fineplot(1:size_S_openmpi(1),S_openmpi(:,5),"Position 5","f","H(f)",[-320 3300],
[-12000 12000],"off",[1500 400],"","-");
subplot(1,3,2)
fineplot(1:size_S_openmpi(1),S_openmpi(:,131),"Position 131","f","H(f)",[-320
3300],[-12000 12000],"off",[1500 400],"","-");
subplot(1,3,3)
fineplot(1:size_S_openmpi(1),S_openmpi(:,1051),"Position 1051","f","H(f)",[-320
3300],[-12000 12000],"off",[1500 400],"","-");
set(gcf, 'Units', 'Normalized', 'OuterPosition', [0, 0, 1, 0.5]);
if save_image
    exportgraphics(gcf, strcat("q41_sysmatrows.png"), 'Resolution', 300)
end

```

Question 4.2

```

[U_openmpi,Sigma_openmpi,V_openmpi] = svd(S_openmpi,'econ');
cond_num_openmpi = Sigma_openmpi(1,1)/Sigma_openmpi(end,end)
s_openmpi = diag(Sigma_openmpi);
figure();
subplot(1,2,1)
p = fineplot(1:numel(s_openmpi),s_openmpi,"Singular Values of
the System Matrix","i","\sigma_i",[-330 3500],[-Sigma_openmpi(1,1)*0.3
Sigma_openmpi(1,1)],"off",[1200 400],"","-");
p.LineWidth = 2;
subplot(1,2,2)
p = fineplot(1:numel(s_openmpi),10*log10(s_openmpi),"Singular Values of
the System Matrix","i","10log(\sigma_i)",[-330 3500],[-70 90],"off",[1200
400],"","-");
p.LineWidth = 2;
set(gcf, 'Units', 'Normalized', 'OuterPosition', [0, 0, 0.9, 0.45]);
if save_image
    exportgraphics(gcf, strcat("q42_sigma.png"), 'Resolution', 300)
end

```

Question 4.3

```

S_reg_openmpi = [S_openmpi eye(size_S_openmpi(1))];
u_reg_openmpi = [u_openmpi ; zeros(size_S_openmpi(1),1)];
lambda = s_openmpi(1)*s_openmpi(end);
c = modified_kaczmarz(S_reg_openmpi,u_reg_openmpi,lambda);
ima = reshape(c,37,37,37);
figure();
montage(reshape(ima,[37, 37, 1, 37]),'displayRange',[0]);
title(strcat("OpenMPI Phantom, Regularized Kaczmarz Recon, $\lambda = \sigma_1
\sigma_N$ = ",num2sci(lambda,"latex")), "Interpreter","latex")
set(gcf, 'Units', 'Normalized', 'OuterPosition', [0, 0, 1, 1]);
if save_image
    exportgraphics(gcf, strcat("q43_phantom.png"), 'Resolution', 300)
end

```

Question 4.4

```

lambda_rels2 = [10^-6 10^-5 10^-4 10^-3 10^-2];

```

```

for i = 1:numel(lambda_rels2)
    c = modified_kaczmarz(S_reg_openmpi,u_reg_openmpi,lambda_rels2(i) *
(Sigma_openmpi(1,1)^2));
    ima = reshape(c,37,37,37);
    ima(ima<0) = 0;
    figure();
    montage(reshape(ima,[37, 37, 1, 37]),'displayRange',[0,0]);
    title(strcat("OpenMPI Phantom, Regularized Kaczmarz Recon,  $\lambda =$ 
\sigma_1^2 10^{",num2str(i-7),"}"), "Interpreter","latex")
    set(gcf, 'Units', 'Normalized', 'OuterPosition', [0, 0, 1, 1]);
    if save_image

exportgraphics(gcf,strcat("q44",num2str(i),"_phantom.png"), 'Resolution',300)
end
end

```

Functions

```

function c = modified_kaczmarz(S,u,lambda)
    num_iter = 10;

    [m,n] = size(S);

    c = zeros(n,1); % image / concentration vector
    v = zeros(m,1); % auxiliary vector
    sqrt_lambda = sqrt(lambda);

    for iter = 1:num_iter
        p = randperm(m); % randomized row order

        for k = 1:m
            i = p(k);

            s_i = S(i,:).'; % i-th row, written as column vector

            alpha = (u(i) - (S(i,:)*c) - sqrt_lambda*v(i))/ (sum(S(i,:).^2) +
lambda);

            c = c + alpha * s_i;
            v(i) = v(i) + alpha * sqrt_lambda;
        end

        c(c < 0) = 0;
    end
    c = c(1:37^3);
end

function c = kaczmarz_method(S,u,square_side,lambda,plot_configs)
%merged function for both regularised and standard. This way of seperating
%via if else clauses is completely redundant mathematically as selecting
%lambda as 0 already converts the formula to the standard case. However
%this seperation was necessary for labelling images and files etc.
arguments

```

```

S
u
square_side = 50 %side length of image
lambda = 0
plot_configs = {0,0,0,0,"",[]}
end

size_S = size(S);
c = zeros(size_S(2),1);
num_iter = 10;

if plot_configs{1}
    PSNRs = zeros(1,num_iter);
    SSIMs = zeros(1,num_iter);
end

for iter = 1:num_iter
    p = randperm(size_S(1));
    for i = 1:size_S(1)
        if lambda
            n = size_S(2) - size_S(1); % original number of unknowns in c
            alpha = (u(p(i)) - sum(ctranspose(S(p(i),1:n)).*c(1:n)) -
sqrt(lambda)*c(n + p(i))) / (sum(S(p(i),1:n).^2) + lambda);
            c(1:n) = c(1:n) + alpha * ctranspose(S(p(i),1:n));
            c(n + p(i)) = c(n + p(i)) + alpha * sqrt(lambda);
        else
            residual = u(p(i)) - sum(ctranspose(S(p(i),:)).*c);
            c = c + (residual / sum(S(p(i),:).^2)) * ctranspose(S(p(i),:));
        end
    end
end
if lambda
    c(1:square_side^2) = max(c(1:square_side^2),0);
else
    c = max(c,0);
end
if plot_configs{1}
    ima = reshape(c(1:square_side^2),square_side,square_side);
    ima(ima<0) = 0;
    PSNRs(iter) = psnr(ima/plot_configs{3},plot_configs{4}/
plot_configs{3});
    SSIMs(iter) = ssim(ima/plot_configs{3},plot_configs{4}/
plot_configs{3});
    if lambda
        plot_comparetive_images(ima,plot_configs{4},plot_configs{2},sprintf("Q34_comp_ite
r = %i",iter),sprintf("Kaczmarz Recon, %s = %s, Iteration = %i","$
\lambda$",num2sci(lambda,"latex"),iter),"Reference Image",plot_configs{3})
    else
        plot_comparetive_images(ima,plot_configs{4},plot_configs{2},sprintf("Q32_comp_ite
r = %i",iter),sprintf("Kaczmarz Recon, Threshold = %s, Iteration =
%i",plot_configs{5},iter),"Reference Image",plot_configs{3})
    end
end

```

```

        end
    end
    if plot_configs{1}
        figure();
        subplot(1,2,1)
        fineplot(1:num_iter,SSIMs,"SSIM Values for Each
Iteration","Iteration","SSIM",[-1 11],[-0.1 1.2],"off",[800 400],"","-");
        subplot(1,2,2)
        fineplot(1:num_iter,PSNRs,"PSNR Values for Each
Iteration","Iteration","PSNR",[-1 11],[min(PSNRs)-1 max(PSNRs)+1],"off",[800
400],"","-");
        set(gcf, 'Units', 'Normalized', 'OuterPosition', [0, 0.04, 0.8, 0.4]);
        if plot_configs{2}
            if lambda
                exportgraphics(gcf,strcat("q34_ssimsnr.png"),'Resolution',300)
            else
                exportgraphics(gcf,strcat("q32_ssimsnr.png"),'Resolution',300)
            end
        end
    end
    end
    c = c(1:square_side^2);
end

function out = num2sci(num, interpreter, precision)
    arguments
        num (1,1) double
        interpreter (1,1) string = "none"
        precision (1,1) double = 3
    end

    if num == 0
        if interpreter == "latex"
            out = "$0$";
        else
            out = "0";
        end
        return;
    end

    order = floor(log10(abs(num)));
    coeff = num / (10^order);

    if interpreter == "latex"
        out = sprintf("$%. *f \\times 10^{\\%d}$", precision, coeff, order);
    else
        out = sprintf("%. *f x 10^{\\%d}", precision, coeff, order);
    end
end

function sig_inv = invert_sigma(Sigma)
    s_func = diag(Sigma);
    sinv = zeros(size(s_func));
    tol = 1e-12; % or use a tolerance based on max(s)
    sinv(s_func > tol) = 1 ./ s_func(s_func > tol);
end

```



```

    sig_inv = diag(sinv);
end

function
plot_comparative_images(im1,ref,save_image,save_str,compared_str,ref_str,max_val,
interval)
    arguments
        im1
        ref
        save_image
        save_str
        compared_str
        ref_str = "Reference Image"
        max_val = 1
        interval = [0 max_val/2]
    end
    figure;
    subplot(1,3,1)
    imshow(im1,[0 max_val])
    % colorbar
    title(compared_str,Interpreter="latex")
    subplot(1,3,2)
    imshow(ref,[0 max_val])
    % colorbar
    title(ref_str,Interpreter="latex")
    subplot(1,3,3)

    imshow(abs(abs(im1) - abs(ref)),interval)
    % colorbar

    title("Error Map",Interpreter="latex")
    set(gcf,'position',[0, 0, 1500 , 450]);
    set(gcf, 'Units', 'Normalized', 'OuterPosition', [0, 0.04, 0.7, 0.7]);
    if save_image
        exportgraphics(gcf,strcat(save_str,".png"),'Resolution',300)
    end
end

%x,y are the variables to be plotted.
%titlename,axisname,yaxisname,legendname are the labels in their respective
areas
%size is in pixels
%holdstate = 'on' is equivalent to "hold on"
%xlimits and ylimits are xlim and ylim
%style is the specifying string for color linestyle etc. like 'k--'
function plot_n =
fineplot(x,y,titlename,axisname,yaxisname,xlimits,ylimits,holdstate,size,legendN
ame,style)
    arguments
        x
        y
        titlename string = ''
        axisname string = ''
        yaxisname string = ''

```

```

    xlimits (1,2) double = [min(x) max(x)]
    ylimits (1,2) double = [min(y) max(y)]
    holdstate string = 'off'
    size (1,2) double = [400 400];
    legendName string = ''
    style = '-'
end
% x = [x(1) x];
% y = [y(1) y];
%even if the limits are reverse this section corrects the order
if ylimits(1) > ylimits(2)
    ylimits = [ylimits(2) ylimits(1)];
end
if xlimits(1) > xlimits(2)
    xlimits = [xlimits(2) xlimits(1)];
end

%these measures are necessary for positioning arrows, labels etc.
dx = ((xlimits(2)-xlimits(1))/65);
dy = ((ylimits(2)-ylimits(1))/85);

%this section is to not plot values that overlap with the arrows
% lowindex = 1;
% highindex = length(x);
% for i = x
%     if (i > (xlimits(1) + dx)) & (lowindex == 1)
%         lowindex = find(x==i);
%     end
%     if (i > xlimits(2) - dx) & (highindex == 1)
%         highindex = find(x==i)-1;
%     end
% end
% if x(lowindex) > 0 %we allow points to be drawn at the origin
%     lowindex = lowindex - 1;
% end
%
% if x(highindex) < 0 %we allow points to be drawn at the origin
%     highindex = highindex + 1;
% end
%
% x = x(lowindex:highindex);
% y = y(lowindex:highindex);

%this section is responsible for axis configuration

plot_n = plot(x,y,style,'displayName',legendName);
xlim(xlimits);
ylim(ylimits);
set(get(gca,'XLabel'),'Visible','on');
set(gca,'XAxisLocation','origin','box','off');
set(gca,'YAxisLocation','origin');
set(get(gca,'XAxis'),'FontWeight','bold');
set(get(gca,'YAxis'),'FontWeight','bold');

```

```

set(get(gca, 'YLabel'), 'Visible', 'on');
set(gca, 'Layer', 'top');
set(gcf, 'position', [0, 0, size(1) , size(2)]);

%deletes the ticks that overlap with arrows
xticks('auto');
xt = xticks;
xticks(xt(2:length(xt)-1));
yticks('auto');
yt = yticks;
yticks(yt(2:length(yt)-1));

% determining the ylevel to draw the arrows on the x axis
if ((ylimits(1) * ylimits(2)) < 0)
    xal = 0;
elseif ylimits(1) < 0
    xal = ylimits(2);
else
    xal = ylimits(1);
end

%plotting the arrows
hold on
if xlimits(2) > 0
    plot([(xlimits(2)-dx) xlimits(2) (xlimits(2)-dx)], [xal+dy xal xal-
dy], 'k', 'HandleVisibility', 'off') %xaxis right arrow
end
if xlimits(1) < 0
    plot([xlimits(1)+dx xlimits(1) xlimits(1)+dx], [xal+dy xal xal-
dy], 'k', 'HandleVisibility', 'off') %xaxis left arrow
end
plot([0 0], [ylimits(2) ylimits(1)], 'k', 'HandleVisibility', 'off') % y axis
if ylimits(2) > 0
    plot([-dx/2 0 dx/2], [ylimits(2)-dy ylimits(2) ylimits(2)-
dy], 'k', 'HandleVisibility', 'off') %yaxis top arrow
end
if ylimits(1) < 0
    plot([-dx/2 0 dx/2], [ylimits(1)+dy ylimits(1)
ylimits(1)+dy], 'k', 'HandleVisibility', 'off') %yaxis bottom arrow
end

%repositioning title & label locations
label_h1 = xlabel(xaxisname);
label_h1.Position(1) = xlimits(2)+dx; % change horizontal position of xlabel.
label_h1.Position(2) = dy; % change vertical position of xlabel.c5
label_h2 = ylabel(yaxisname, rotation=0);
label_h2.Position(1) = dx; % change horizontal position of ylabel.
label_h2.Position(2) = ylimits(2)+dy; % change vertical position of ylabel.
title(titlename);
% if ((xlimits(1) * xlimits(2)) < 0) & (xlimits(1)+xlimits(2) < ((xlimits(2)-
xlimits(1))/3))
%     set(get(gca, 'title'), 'Position', [20*dx ylimits(2)-dy]) %prevents the
title from colliding with ylabel
% end

```

```

grid on
%holds or doesnt hold
if strcmp(holdstate,'off')
    hold off
else
    hold on
end
end

%x,y are the variables to be plotted.
%titlename,xaxisname,yaxisname,legendname are the labels in their respective
areas
%size is in pixels
%holdstate = 'on' is equivalent to "hold on"
%xlimits and ylimits are xlim and ylim
%bodestate plots the complex valued array in a bode plot with its phase and
magnitute
%style is the specifying string for color linestyle etc. like 'k--'
function plot_n =
finesemilogx(x,y,bodeState,titlename,xaxisname,yaxisname,xlimits,ylimits,holdstat
e,size,legendname,style)
    arguments
        x
        y
        bodeState = 'off'
        titlename string = ''
        xaxisname string = ''
        yaxisname string = ''
        xlimits (1,2) double = [min(x) max(x)]
        ylimits (1,2) double = [min(y) max(y)]
        holdstate string = 'off'
        size (1,2) double = [400 400];
        legendname string = ''
        style = 'k'
    end
    xlimits = 10.^xlimits;

    %even if the limits are reverse this section corrects the order
    if ylimits(1) > ylimits(2)
        ylimits = [ylimits(2) ylimits(1)];
    end
    if xlimits(1) > xlimits(2)
        xlimits = [xlimits(2) xlimits(1)];
    end

    if strcmp(bodeState,'on') || strcmp(bodeState,'On')
        semilogx(x,abs(y),'DisplayName',strcat(legendname,'Magnitute'))
        hold on
        semilogx(x,angle(y),'--','DisplayName','Phase')
    else
        semilogx(x,y,style,'DisplayName','legendname')
    end
    title(titlename)

```

```

xlabel(xaxisname)
ylabel(yaxisname)
xlim(xlimits)
ylim(ylimits)

%annotation(gcf,'arrow',[0.13 0.941666666666667],[0.11 0.111666666666667]);
%annotation(gcf,'arrow',[0.130729166666667 0.130208333333333],
[0.112347052280311 0.946607341490545]);

xticks('auto')
yticks('auto')

dx = 10^((log10(xlimits(2)) - log10(xlimits(1))) * 0.1 );
dy = ((ylimits(2)-ylimits(1))/85);

label_h1 = xlabel(xaxisname);
label_h1.Position(1) = xlimits(2)*(dx)*0.05; % change horizontal position of
xlabel.
label_h1.Position(2) = ylimits(1)+(dy)*0.7; % change vertical position of
xlabel.c5
label_h2 = ylabel(yaxisname,rotation=0);
label_h2.Position(1) = xlimits(1)*(dx)*0.1; % change horizontal position of
ylabel.
label_h2.Position(2) = ylimits(2); % change vertical position of ylabel.

set(get(gca,'XLabel'),'Visible','on')
set(gca,'XAxisLocation','origin','box','off')
set(gca,'YAxisLocation','origin')
set(get(gca,'XAxis'),'FontWeight','bold')
set(get(gca,'YAxis'),'FontWeight','bold');
set(get(gca,'YLabel'),'Visible','on')
set(gca,'Layer','top')
set(gcf,'position',[0 , 0, size(1) , size(2)])

set(get(gca,'title'),'Position', [sqrt(prod(xlimits)) ylimits(2)-dy*0.1])


```
%prevents the title from colliding with ylabel
hold on

dx = dx^(1/4);
dy = dy^(1/4);
plot([xlimits(2)/dx xlimits(2)],[ylimits(1)-dy ylimits(1)],'k')
plot([xlimits(2)/dx xlimits(2) xlimits(2)/dx],[ylimits(1)-dy*0.1 ylimits(1)
ylimits(1)+dy*0.1],'k')
if strcmp(holdstate,'off')
 hold off
else
 hold on
end

```


```

```

    grid on
end
    %x,y are the variables to be plotted.
    %titlename,xaxisname,yaxisname,legendname are the labels in their respective
    areas
    %size is in pixels
    %holdstate = 'on' is equivalent to "hold on"
    %xlims and ylims are xlim and ylim
    %bodestate plots the complex valued array in a bode plot with its phase and
    magnitude
    %style is the specifying string for color linestyle etc. like 'k--'
function plot_n =
finelloglog(x,y,bodeState,titlename,xaxisname,yaxisname,xlims,ylims,holdstate,
size,legendname,style)
    arguments
        x
        y
        bodeState = 'off'
        titlename string = ''
        xaxisname string = ''
        yaxisname string = ''
        xlims (1,2) double = [min(x) max(x)]
        ylims (1,2) double = [min(y) max(y)]
        holdstate string = 'off'
        size (1,2) double = [400 400];
        legendname string = ''
        style = 'k'
    end
    xlims = 10.^xlims;
    ylims = 10.^ylims;

    %even if the limits are reverse this section corrects the order
    if ylims(1) > ylims(2)
        ylims = [ylims(2) ylims(1)];
    end
    if xlims(1) > xlims(2)
        xlims = [xlims(2) xlims(1)];
    end

    if strcmp(bodeState,'on') || strcmp(bodeState,'On')
        loglog(x,abs(y^20),'DisplayName',strcat(legendname,'Magnitude'))
        hold on
        loglog(x,angle(y),'--','DisplayName','Phase')
    else
        loglog(x,y,style,'DisplayName','legendname')
    end
    title(titlename)
    xlabel(xaxisname)
    ylabel(yaxisname)
    xlim(xlims)
    ylim(ylims)

    %annotation(gcf,'arrow',[0.13 0.941666666666667],[0.11 0.111666666666667]);

```

```

%annotation(gcf,'arrow',[0.130729166666667 0.130208333333333],
[0.112347052280311 0.946607341490545]);

xticks('auto')
yticks('auto')


dx = 10^((log10(xlimits(2)) - log10(xlimits(1))) * 0.1 );
dy = 10^((log10(ylimits(2)) - log10(ylimits(1))) * 0.1 );

label_h1 = xlabel(xaxisname);
label_h1.Position(1) = xlimits(2)*(dx)*0.05; % change horizontal position of
xlabel.
label_h1.Position(2) = ylimits(1)*(dy)*0.75; % change vertical position of
xlabel.c5
label_h2 = ylabel(yaxisname,rotation=0);
label_h2.Position(1) = xlimits(1)*(dx)*2; % change horizontal position of
ylabel.
label_h2.Position(2) = ylimits(2); % change vertical position of ylabel.


set(get(gca,'XLabel'),'Visible','on')
set(gca,'XAxisLocation','origin','box','off')
set(gca,'YAxisLocation','origin')
set(get(gca,'XAxis'),'FontWeight','bold')
set(get(gca,'YAxis'),'FontWeight','bold');
set(get(gca,'YLabel'),'Visible','on')
set(gca,'Layer','top')
set(gcf,'positio',[(xlimits(2)-xlimits(1))/2 , (ylimits(2)-ylimits(1))/2 ,
size(1) , size(2)])


set(get(gca,'title'),'Position', [sqrt(prod(xlimits)) ylimits(2)*(dy)])


```